

MINI CURSO DE GODOT 4

AstroDash

Do zero ao jogo de sobrevivência no espaço, em 2 aulas

Godot 4.7 · GDScript · 2 × 1h20

Use ← → ou clique para navegar · M índice · F tela cheia

INTRODUÇÃO

O que vamos construir

- Um jogo de **sobrevivência** no espaço: asteroides vêm de **todas as direções**.
- A nave desvia; o objetivo é **sobreviver o máximo de tempo**.
- Menu inicial, HUD com tempo e recorde, tela de Game Over e dificuldade que aumenta com o tempo.
- Arte provisória: o **ícone do Godot**. Na 2ª parte do curso a gente desenha tudo no **LibreSprite** e troca.



O jogo pronto (com a arte provisória)

INTRODUÇÃO

Plano das aulas

	AULA 1 (1H20)	AULA 2 (1H20)
1	Apresentação do Godot e configurações do projeto	GameManager (Autoload) e HUD
2	A nave: cena, movimento e colisão	Componentes: vida e área de dano
3	A fase: paredes e cena principal	Game Over e reiniciar
4	O asteroide: corpo físico e sinais	Menu inicial
5	O spawner: asteroides vindo de todo lado	Dificuldade, fundo animado e preparação da arte

> **DICA** Se alguém se perder na Aula 1, o projeto **checkpoint-aula1** no repositório começa a Aula 2 do ponto certo.

INTRODUÇÃO

Como o projeto é organizado

Managers

GameManager (Autoload): tempo, recorde, troca de cena e o aviso de game over.

Componentes

HealthComponent e **HurtboxComponent**: peças reutilizáveis que você encaixa em qualquer cena.

Entidades

Player e **Asteroid**: cada uma é uma cena independente.

Sistemas

AsteroidSpawner: cria os asteroides e controla a dificuldade.

Interface

HUD, **GameOver** e **MainMenu**: só ouvem os sinais do GameManager.

Fases

Game junta tudo: fundo, paredes, spawner, player e UI.

POR QUÊ? Cada peça faz **uma coisa só** e conversa por **sinais**. Assim dá para adicionar inimigos, power-ups ou novas fases sem quebrar o resto.

AULA 1

Nave, asteroides e o primeiro jogo rodando

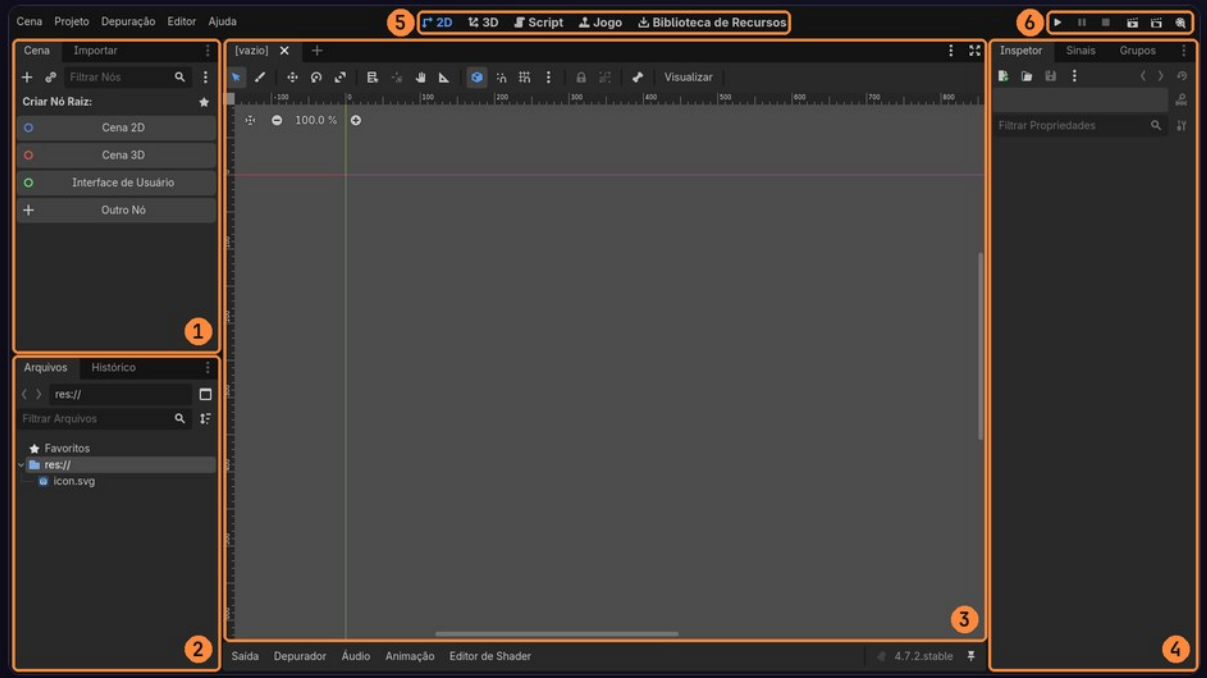
1. Conhecer o editor
2. Configurar o projeto
3. Player
4. Cena Game e paredes
5. Asteroide
6. Spawner

AULA 1 **Etapa 1.1**

Criando o projeto

1. Abra o Godot 4 → **Criar** (Gerenciador de Projetos).
2. Nome: **AstroDash**. Escolha uma pasta vazia.
3. Renderizador: **Forward+** (o padrão serve).
4. Clique em **Criar e Editar**.

> **DICA** Tudo dentro do projeto é acessado pelo caminho `res://` (a pasta do projeto).



1 **Cena**: a árvore de nós da cena aberta

2 **Arquivos**: pastas e arquivos (`res://`)

3 **Viewport**: onde você vê e arrasta as coisas

4 **Inspetor**: propriedades do nó selecionado (também tem **Sinais** e **Grupos**)

5 Telas: **2D**, **3D**, **Script**...

6 Rodar o jogo: `F5` projeto, `F6` cena atual

AULA 1 Etapa 1.2

Nós e cenas em 1 minuto

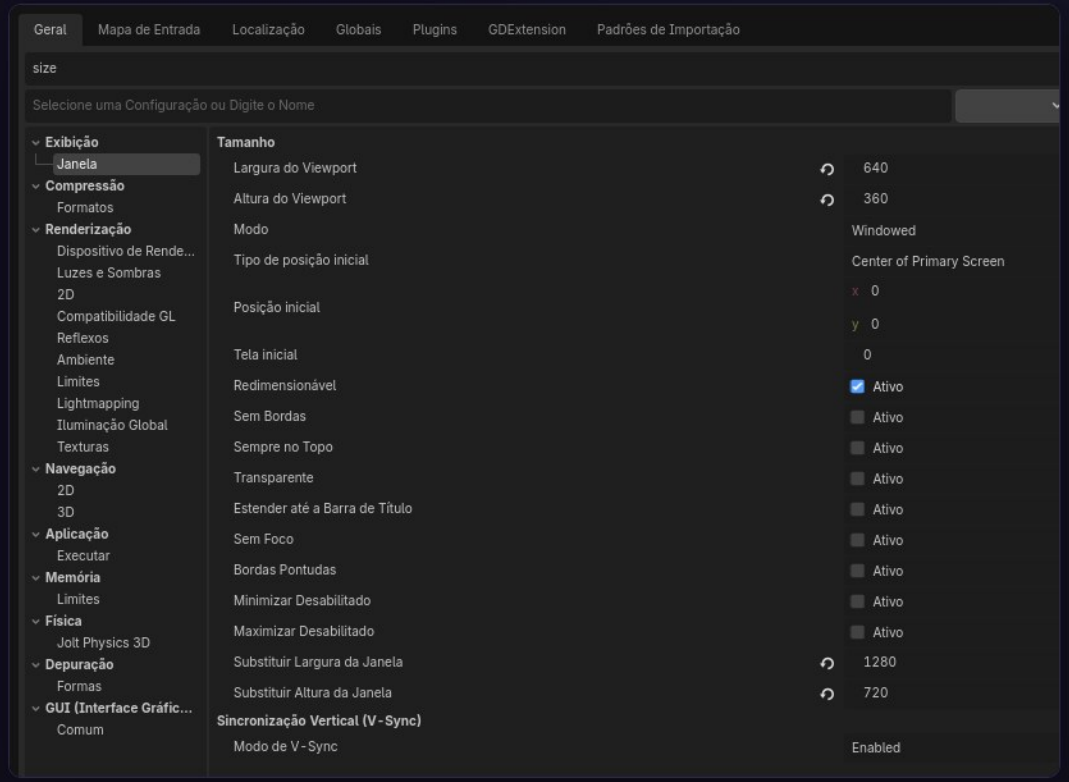
- Tudo no Godot é um **nó** (Node). Cada tipo de nó faz uma coisa: desenhar uma imagem, colidir, tocar som, contar tempo...
- Uma **cena** é uma árvore de nós salva em um arquivo `.tscn`.
- Cenas podem ser **instanciadas** dentro de outras: a cena Game vai conter a cena Player, que por sua vez contém componentes.
- **Sinais** são avisos que um nó emite ("o timer acabou", "o botão foi apertado"). Outros nós se conectam a eles.
- Regra de ouro do curso: **se dá para fazer no editor, faz no editor**. Código só para lógica.

AULA 1 Etapa 1.3

Configurações: tamanho da tela

1. **Projeto → Configurações do Projeto →** aba **Geral**.
2. Ligue **Configurações Avançadas** (canto superior direito).
3. Busque `size` → **Exibição → Janela**:
4. **Largura do Viewport** = 640, **Altura do Viewport** = 360
5. **Substituir Largura da Janela** = 1280, **Altura** = 720

> **POR QUÊ?** O jogo é desenhado em 640×360 (ótimo para pixel art) e a janela abre em 1280×720 (2× maior).



AULA 1 Etapa 1.4

Configurações: esticar e pixel art

1. Busque `stretch` → **Exibição** → **Janela** → **Esticar**:
2. **Modo** = `canvas_items`, **Aspecto** = `keep`, **Modo de Escalonamento** = `integer`
3. Busque `texture_filter` → **Filtro de Textura Padrão** = `Nearest`
4. Busque `clear_color` → **Cor de Fundo Padrão**: um azul bem escuro (ex.: `#0a0819`)

> POR QUÊ? **Nearest** deixa o pixel art nítido (sem borrar) quando a imagem é ampliada.





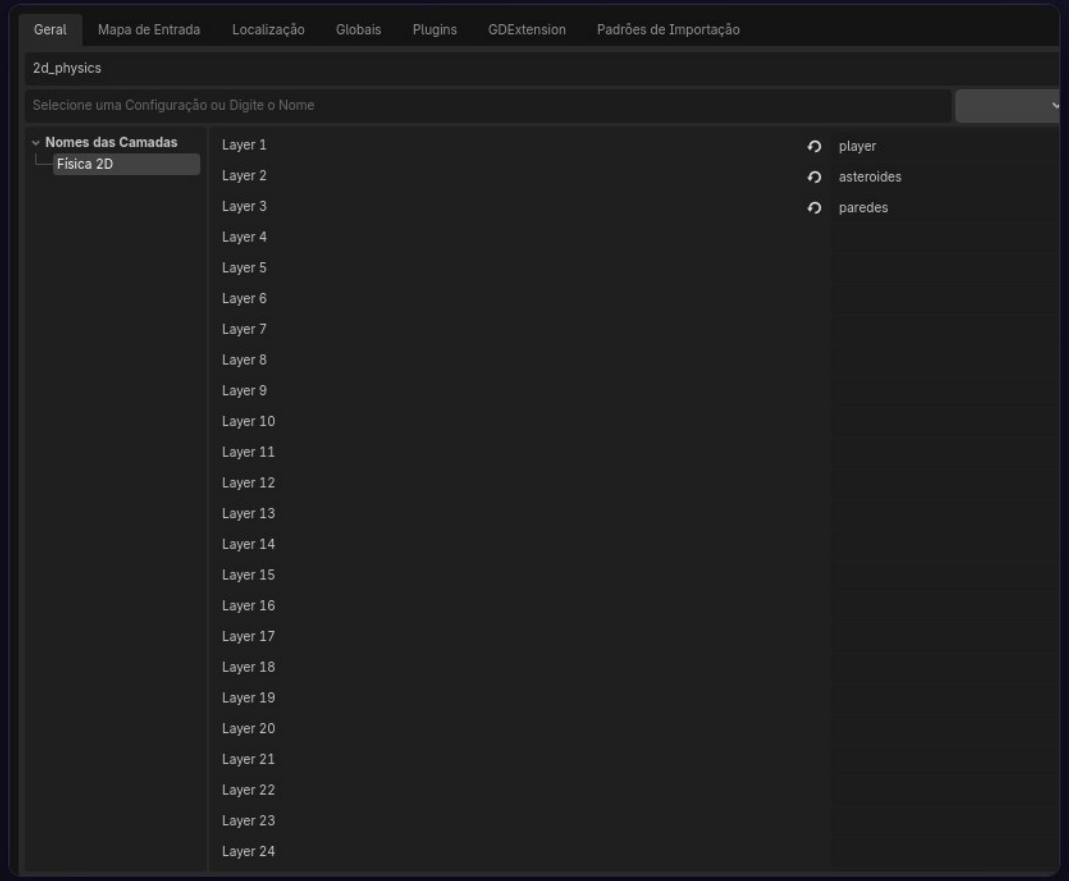
AULA 1 Etapa 1.5

Configurações: nomes das camadas de física

1. Busque `2d_physics` → **Nomes de Camadas → Física 2D**.
2. Layer 1 = `player`, Layer 2 = `asteroides`, Layer 3 = `paredes`

OBJETO	ESTÁ NA CAMADA (LAYER)	ENXERGA (MASK)
Player (corpo)	player	paredes
Hurtbox do Player	nenhuma	asteroides
Asteroide	asteroides	nenhuma
Paredes	paredes	nenhuma

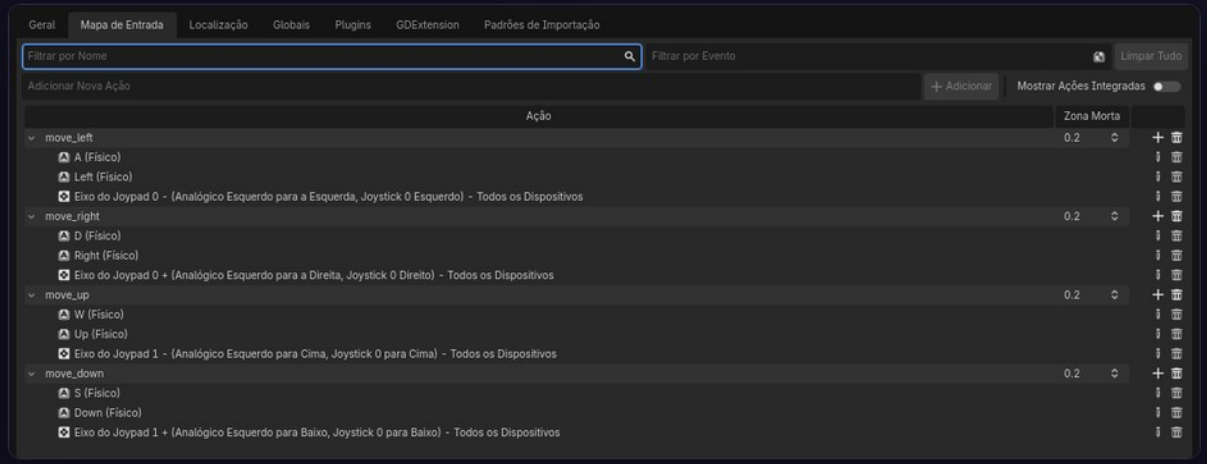
> POR QUÊ? **Layer** = onde eu estou. **Mask** = o que eu detecto. Asteroides atravessam tudo, mas a Hurtbox do player sente quando um encosta.



AULA 1 **Etapa 1.6**

Configurações: Mapa de Entrada

1. Aba **Mapa de Entrada**.
2. Em **Adicionar Nova Ação** crie: `move_left`, `move_right`, `move_up`, `move_down`.
3. Clique no **+** de cada ação e aperte a tecla: **A/D/W/S** e as **setas**.
4. Opcional: adicione o **analógico esquerdo** do controle (Eixo do Joypad).



> **FERRAMENTA NATIVA** O código pede "move_left", não "tecla A". Trocar ou adicionar controles depois não mexe em nenhuma linha de código.

AULA 1 Etapa 1.7

Estrutura de pastas

No painel **Arquivos**: botão direito em `res://` → **Nova Pasta**.

PASTA	O QUE GUARDA
`autoload/`	Managers globais (GameManager)
`components/`	Componentes reutilizáveis
`entities/player/`	Cena e script da nave
`entities/asteroid/`	Cena e script do asteroide
`systems/asteroid_spawner/`	Gerador de asteroides
`levels/game/`	A fase principal
`levels/background/`	Fundo animado
`ui/`	HUD, Game Over, Menu e tema
`assets/sprites/`	Artes (2ª parte do curso)

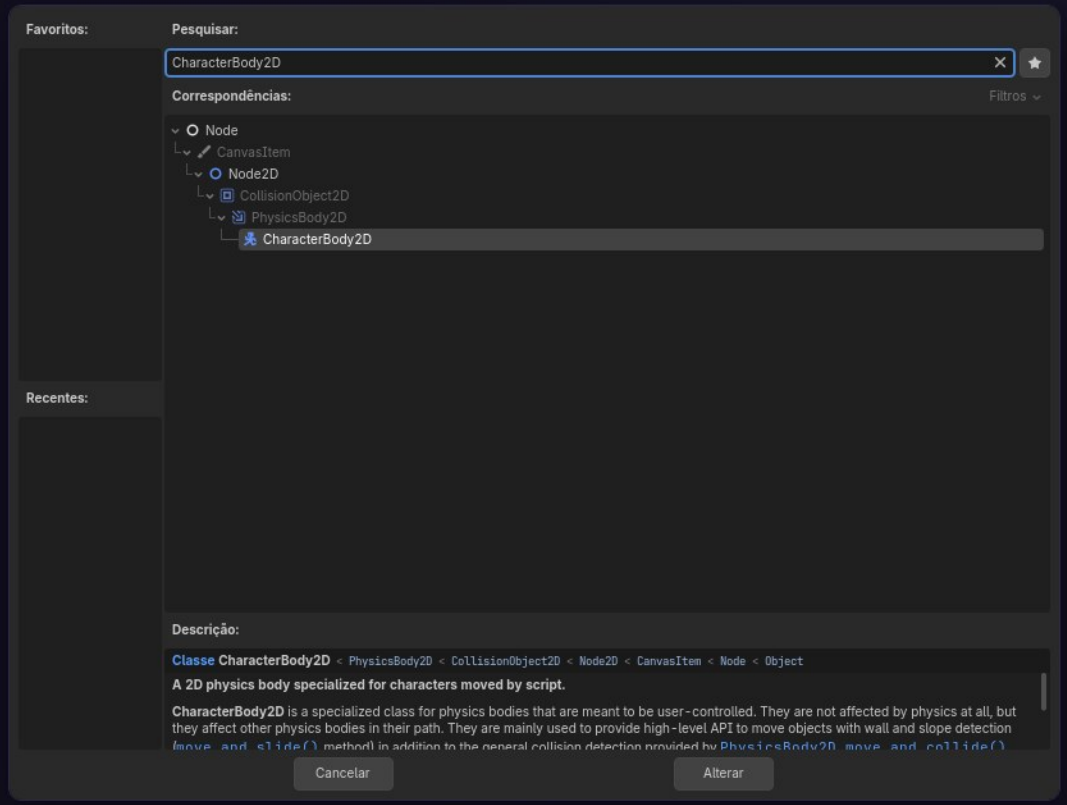
> **DICA** Cada cena fica numa pasta junto com seu script. Fica fácil achar e reaproveitar.

AULA 1 Etapa 2.1

Player: criando a cena

1. **Cena → Nova Cena → Outro Nó**.
2. Busque **CharacterBody2D** e clique em **Criar**.
3. Renomeie para **Player** (`F2`).
4. Com o Player selecionado, `Ctrl` + `A` (Adicionar Nó Filho): **Sprite2D**.
5. De novo: **CollisionShape2D**.
6. Salve (`Ctrl` + `S`) em `entities/player/player.tscn`

> **POR QUÊ?** **CharacterBody2D** é o nó feito para personagens controlados pelo jogador: ele se move com `move_and_slide()` e para nas paredes sozinho.



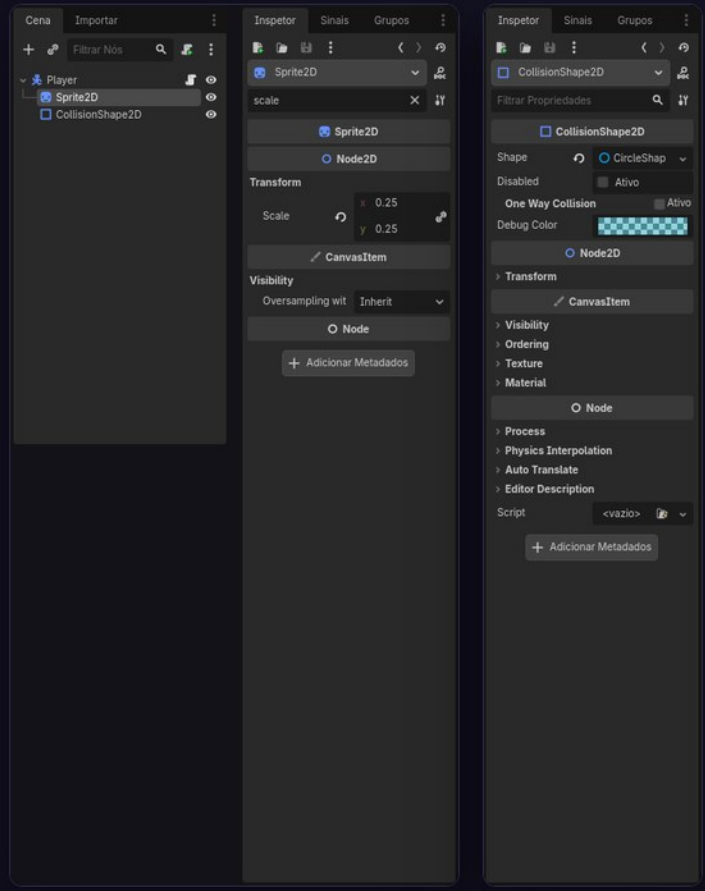
Janela Criar Novo Nó

AULA 1 Etapa 2.2

Player: sprite e colisão

1. Selecione **Sprite2D** → arraste o `icon.svg` dos Arquivos para **Texture**.
2. Em **Transform** → **Scale** coloque `0.25` (o ícone tem 128px → fica 32px).
3. Selecione **CollisionShape2D** → **Shape** → **Novo CircleShape2D**.
4. Clique no círculo e ajuste **Radius** = `12` (um pouco menor que a nave = jogo mais justo).

> **ATENÇÃO** Desenhe a nave da 2ª parte **apontando para a direita**. No Godot, ângulo 0 = direita.

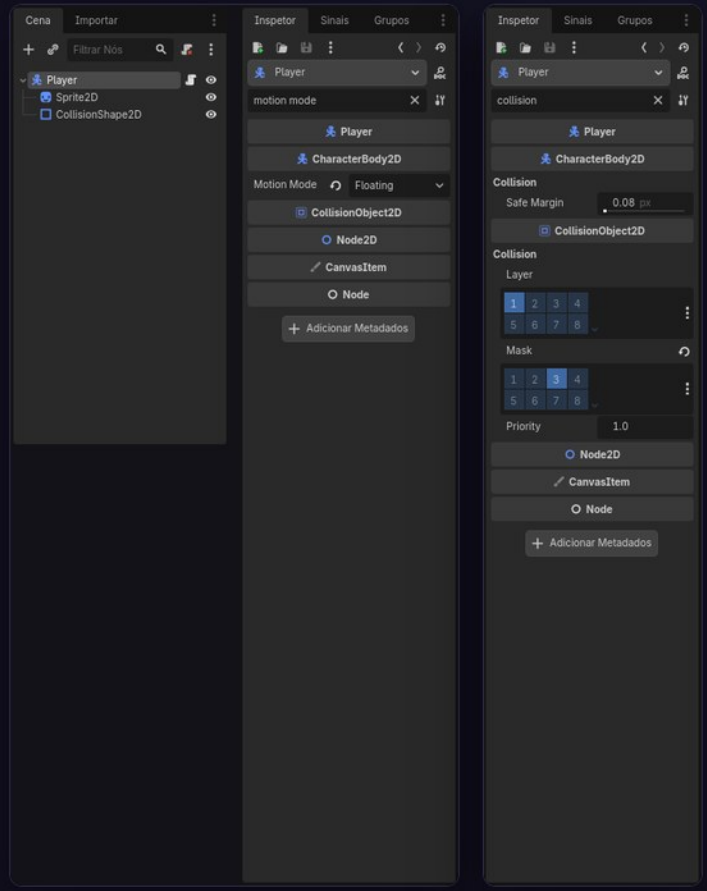


AULA 1 Etapa 2.3

Player: propriedades no Inspetor

1. Selecione o nó **Player**.
2. **Motion Mode** = `Floating` (jogo visto de cima, sem chão).
3. **Collision → Layer**: só a **1** (player).
4. **Collision → Mask**: só a **3** (paredes).

> **DICA** Passe o mouse sobre os quadradinhos de Layer/Mask: aparecem os nomes que demos (player, asteroides, paredes).



AULA 1 Etapa 2.4

Player: script de movimento

Selecione o Player → botão **Anexar Script** (pergaminho com +) → salve em `entities/player/player.gd`. `@export` faz a variável aparecer no Inspetor: dá para ajustar a velocidade sem abrir o código.

entities/player/player.gd Copiar

```
class_name Player
extends CharacterBody2D


## Velocidade máxima da nave (pixels por segundo).
@export var speed: float = 180.0
## Quão rápido a nave acelera e freia.
@export var acceleration: float = 900.0
## Quão rápido a nave gira para a direção do movimento.
@export var rotation_speed: float = 10.0


func _physics_process(delta: float) -> void:
    var direction := Input.get_vector("move_left", "move_right", "move_up", "move_down")
    velocity = velocity.move_toward(direction * speed, acceleration * delta)
    move_and_slide()

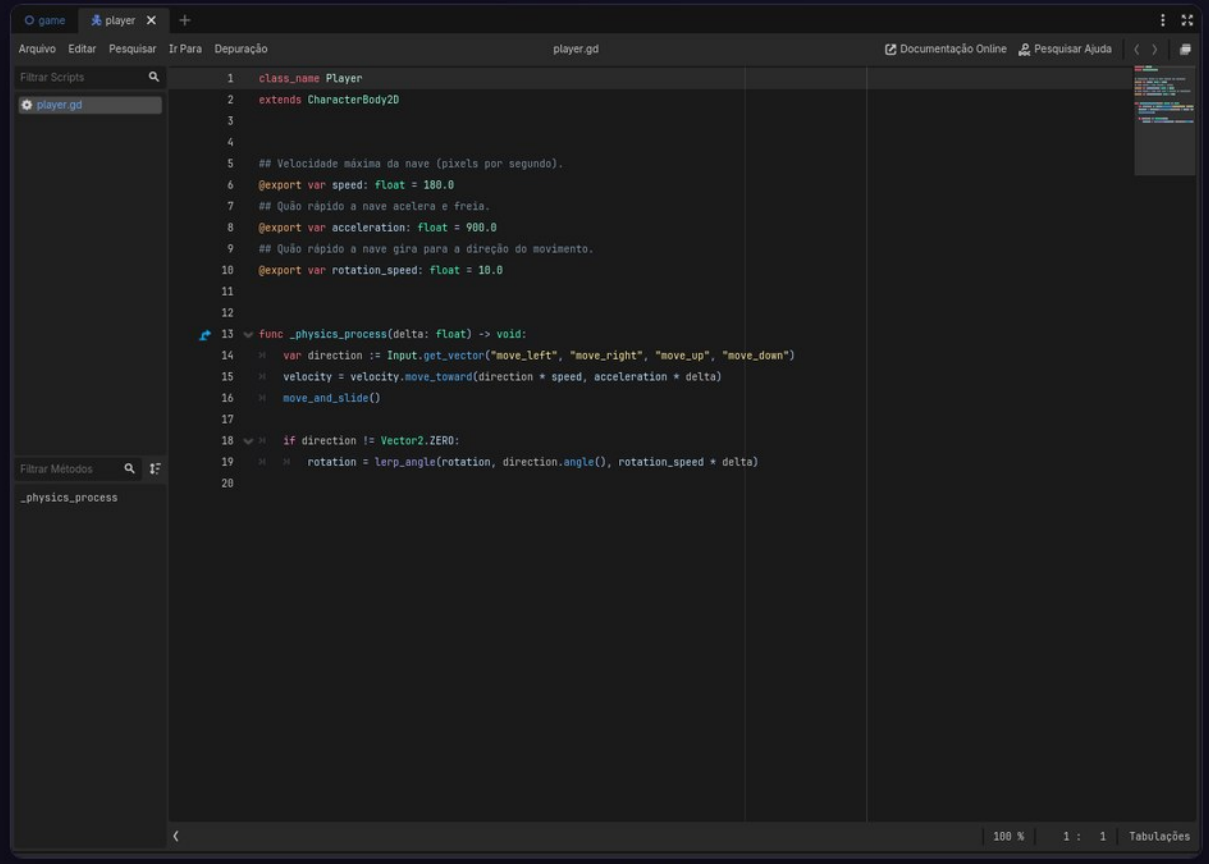
    if direction != Vector2.ZERO:
        rotation = lerp_angle(rotation, direction.angle(), rotation_speed * delta)
```

AULA 1 Etapa 2.5

Entendendo o movimento

- `Input.get_vector(...)` junta as 4 ações num vetor de direção (já normalizado, funciona com analógico).
- `velocity.move_toward(alvo, aceleração * delta)` acelera e freia suavemente, dando sensação de nave.
- `move_and_slide()` move o corpo e respeita colisões (as paredes).
- `lerp_angle` gira a nave aos poucos para onde ela está indo.
- `delta` = tempo desde o último frame. Multiplicar por delta deixa o jogo igual em qualquer PC.

> **DICA** Mude **Speed**, **Acceleration** e **Rotation Speed** no Inspetor com o jogo rodando e sinta a diferença.



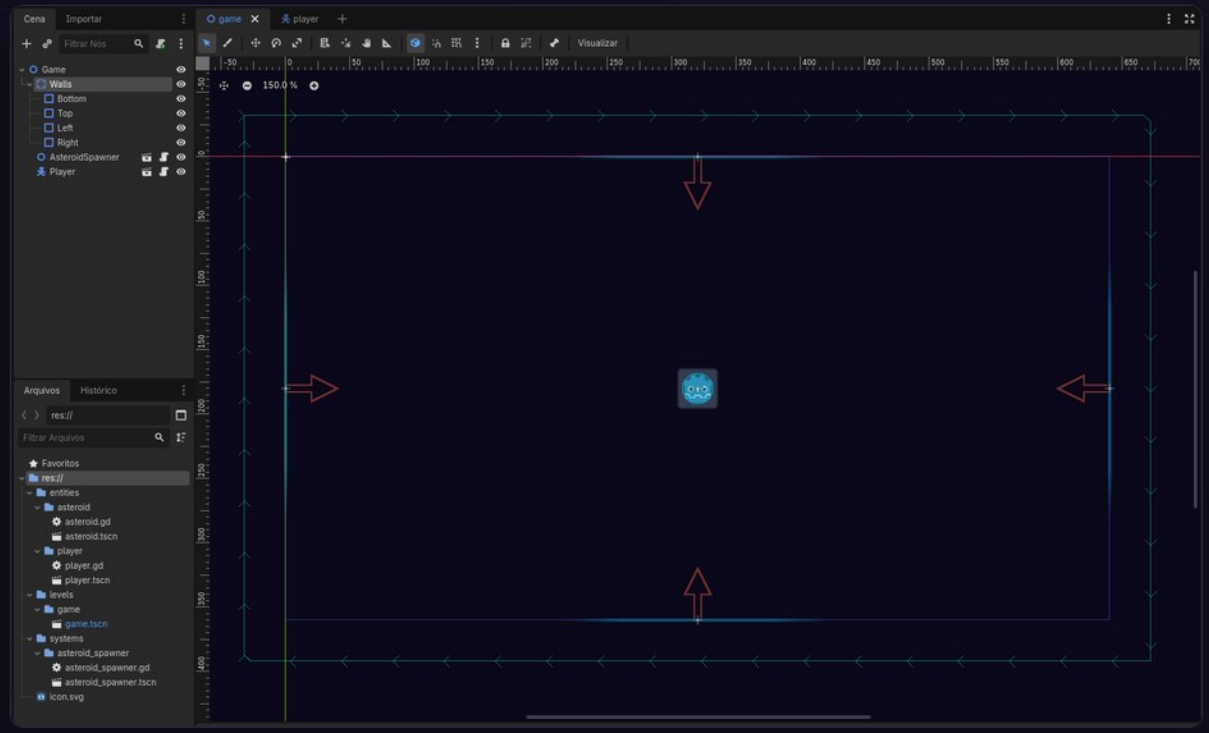
Script aberto na tela Script

AULA 1 **Etapa 3.1**

Cena Game e as paredes

1. **Nova Cena** → **Cena 2D** (Node2D) → renomeie **Game**. Salve em `levels/game/game.tscn`
2. Adicione um **StaticBody2D** chamado **Walls**. Layer = **3** (paredes), Mask = nenhuma.
3. Dentro dele, 4× **CollisionShape2D** com Shape **WorldBoundaryShape2D** (Bottom, Top, Left, Right).
4. Posicione cada uma (tabela ao lado). A seta vermelha aponta para **dentro** da tela.

NÓ	POSITION	ROTATION
Bottom	(320, 360)	0°
Top	(320, 0)	180°
Left	(0, 180)	90°
Right	(640, 180)	-90°



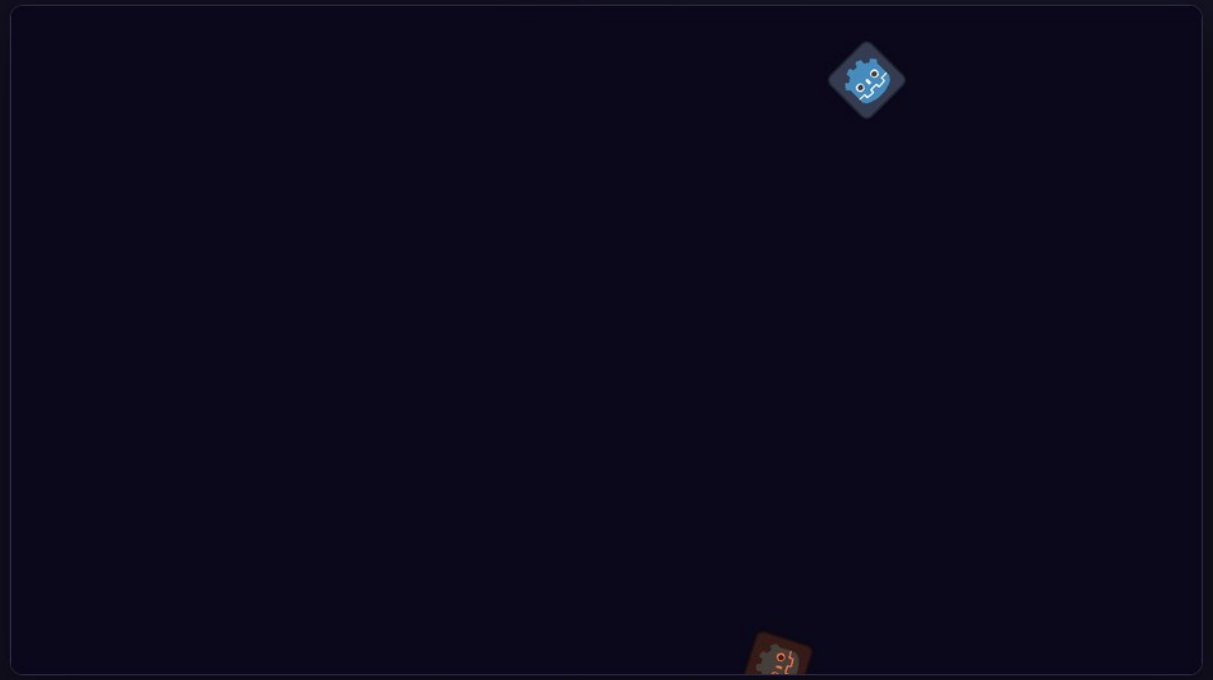
FERRAMENTA NATIVA **WorldBoundaryShape2D** é uma parede infinita. Dá para criar a primeira e depois **Duplicar** (`Ctrl` + `D`), usando o **mesmo** shape.

AULA 1 Etapa 3.2

Colocando o Player na fase

1. Arraste `entities/player/player.tscn` dos Arquivos para o viewport da cena Game (ou use o botão **Instanciar Cena Filha**, o elo de corrente).
2. Position do Player = `(320, 180)` (centro da tela).
3. Defina Game como cena principal: botão direito em `game.tscn` → **Definir como Cena Principal**.
4. Aperte `F5` e voe com WASD/setas!

> **DICA** O retângulo azul no viewport é a tela do jogo (640×360).



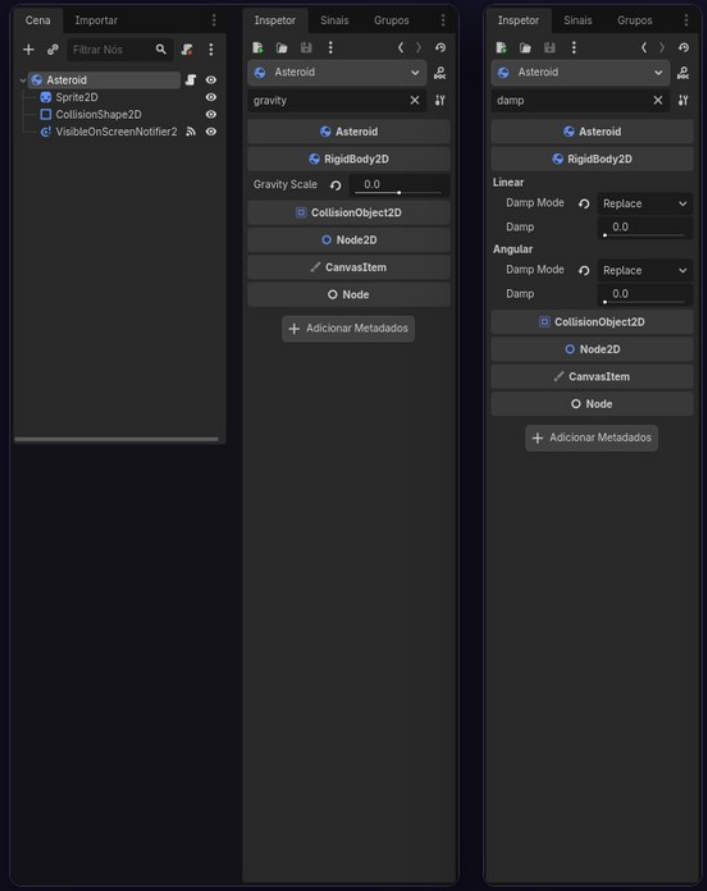
A nave presa dentro das paredes

AULA 1 Etapa 4.1

Asteroide: um corpo físico

1. Nova cena com raiz **RigidBody2D** → renomeie **Asteroid**. Salve em `entities/asteroid/asteroid.tscn`
2. Filhos: **Sprite2D** (icon.svg, Scale 0.25, Modulate laranja), **CollisionShape2D** (círculo, raio 14) e **VisibleOnScreenNotifier2D**.
3. No Asteroid: **Gravity Scale** = `0` (espaço não tem gravidade!).
4. **Linear** → **Damp Mode** = `Replace` e **Angular** → **Damp Mode** = `Replace` (com Damp 0 ele não perde velocidade).
5. **Layer** = 2 (asteroides), **Mask** = nenhuma (atravessa tudo).

> FERRAMENTA NATIVA O **RigidBody2D** é movido pelo motor de física: a gente só dá a velocidade inicial e o Godot faz o resto, inclusive o giro.



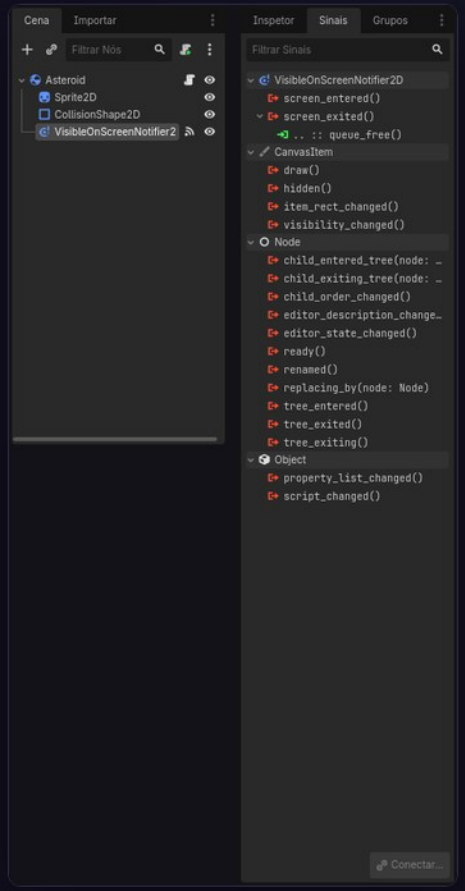
AULA 1 Etapa 4.2

Asteroide: sumir ao sair da tela (sem código!)

1. Selecione **VisibleOnScreenNotifier2D** → aba **Sinais**.
2. Duplo clique em `screen_exited()`.
3. Escolha o nó **Asteroid** e clique em **Escolher** ao lado do Método Receptor.
4. Selecione `queue_free` (desligue "Somente Métodos de Script" se ele não aparecer) → **Conectar**.

POR QUÊ? Sem isso, os asteroides que saem da tela continuariam existindo para sempre e o jogo ficaria lento.

FERRAMENTA NATIVA Conectamos um sinal direto a uma função que já existe em todo nó (`queue_free` = "me apague"). Zero linhas de código.



AULA 1 Etapa 4.3

Asteroide: script

Anexe ao nó raiz **Asteroid** e salve em `entities/asteroid/asteroid.gd`. `class_name Asteroid` registra um tipo novo: outros scripts podem escrever `var a: Asteroid`. Quem chama `launch()` é o spawner, que diz a direção.

entities/asteroid/asteroid.gd — Copiar

```
class_name Asteroid
extends RigidBody2D

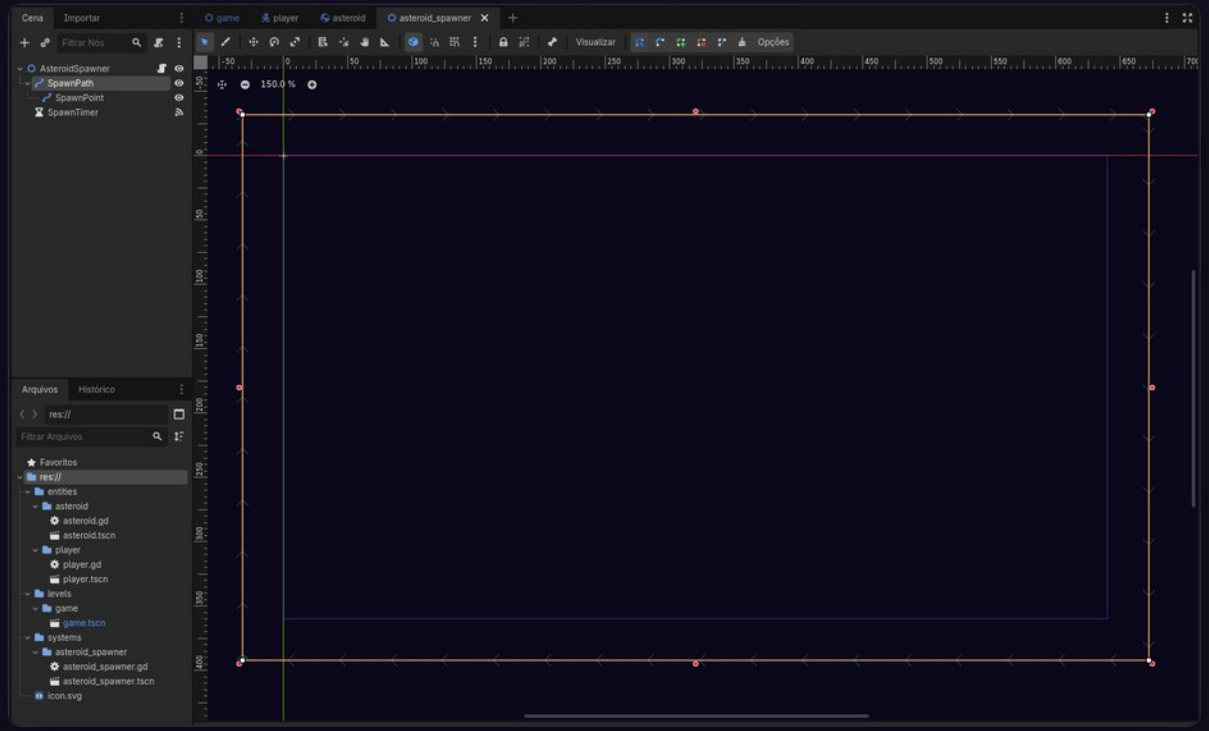

@export var min_speed: float = 60.0
@export var max_speed: float = 120.0
## Velocidade máxima de giro (radianos por segundo).
@export var max_spin: float = 3.0


## Lança o asteroide em uma direção. Quem chama é o AsteroidSpawner.
func launch(direction: Vector2, speed_multiplier: float = 1.0) -> void:
    linear_velocity = direction * randf_range(min_speed, max_speed) * speed_multiplier
    angular_velocity = randf_range(-max_spin, max_spin)
```

AULA 1 Etapa 5.1

Spawner: o caminho em volta da tela

1. Nova cena com raiz **Node2D** → **AsteroidSpawner**. Salve em `systems/asteroid_spawner/asteroid_spawner.tscn`
2. Adicione **Path2D** (renomeie **SpawnPath**).
3. Com ele selecionado, use a ferramenta **Adicionar Ponto** da barra do viewport e clique nos 4 cantos **por fora** do retângulo azul: (-32,-32), (672,-32), (672,392), (-32,392).
4. Clique em **Fechar Curva**.
5. Dentro do SpawnPath, adicione **PathFollow2D** (**SpawnPoint**) e desligue **Rotates**.



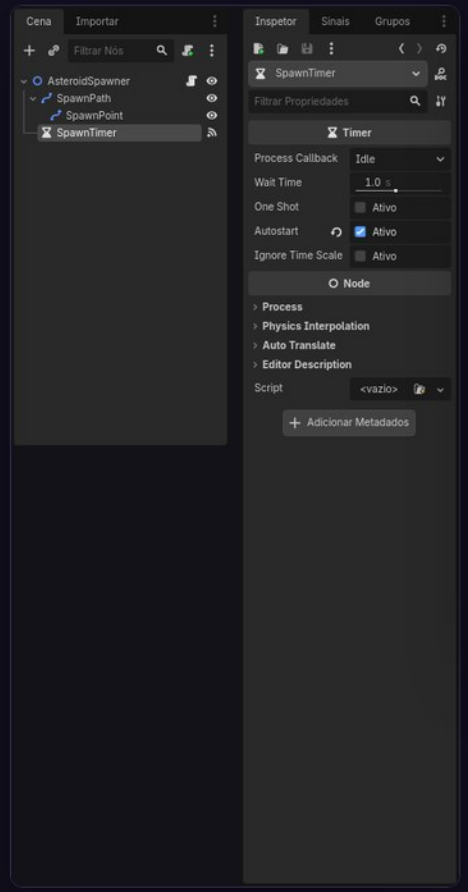
DICA Não precisa ser exato: basta o retângulo ficar um pouco fora da tela, para o asteroide nascer escondido.

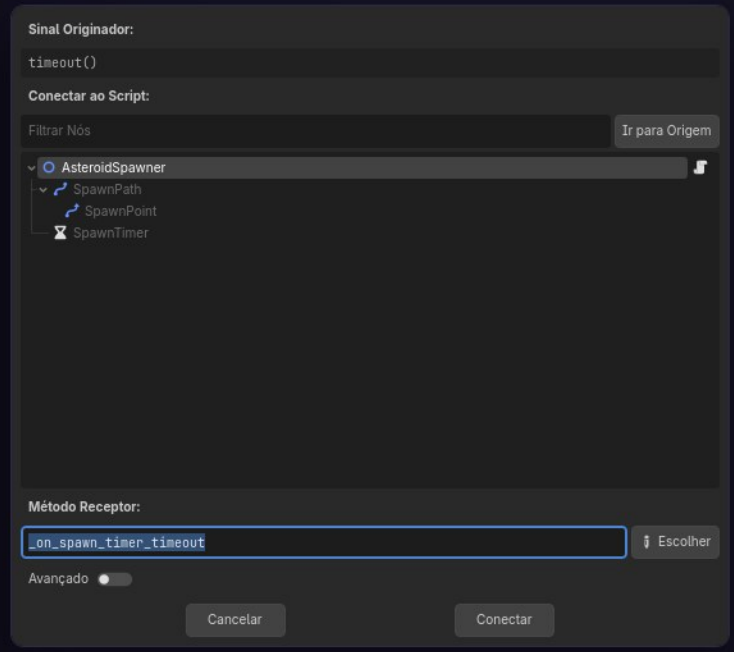
AULA 1 Etapa 5.2

Spawner: o Timer

1. Adicione um **Timer** (renomeie **SpawnTimer**) como filho do AsteroidSpawner.
2. **Wait Time** = 1 e **Autostart** = ligado.
3. Anexe um script vazio ao AsteroidSpawner (próximo slide).
4. Selecione o SpawnTimer → aba **Sinais** → duplo clique em `timeout()` → escolha **AsteroidSpawner** → **Conectar**.

> **FERRAMENTA NATIVA** O Godot cria a função `_on_spawn_timer_timeout` no script para você. O ícone verde ao lado do sinal mostra que ele está conectado.





AULA 1 Etapa 5.3

Spawner: script (versão Aula 1) (1/2)

Depois de salvar, selecione o AsteroidSpawner e arraste `asteroid.tscn` para a propriedade **Asteroid Scene** no Inspetor.

systems/asteroid_spawner/asteroid_spawner.gd Copiar

```
class_name AsteroidSpawner
extends Node2D
## Cria asteroides em volta da tela, cruzando a tela em direções aleatórias.


@export var asteroid_scene: PackedScene

@onready var spawn_point: PathFollow2D = $SpawnPath/SpawnPoint


func _on_spawn_timer_timeout() -> void:
    spawn_point.progress_ratio = randf()
    var asteroid: Asteroid = asteroid_scene.instantiate()
    add_child(asteroid)
    asteroid.global_position = spawn_point.global_position

    var target := _pick_target()
    asteroid.launch(asteroid.global_position.direction_to(target))
```

AULA 1 Etapa 5.3

Spawner: script (versão Aula 1) (2/2)

systems/asteroid_spawner/asteroid_spawner.gd Copiar

```gdscript
## Escolhe um ponto qualquer da tela para o asteroide atravessar.
func _pick_target() -> Vector2:
    var screen := get_viewport_rect().size
    return Vector2(randf_range(0, screen.x), randf_range(0, screen.y))
```

AULA 1 Etapa 5.4

Juntando tudo

1. Abra a cena **Game** e instancie `asteroid_spawner.tscn`.
2. Arraste o AsteroidSpawner para **cima** do Player na árvore (assim a nave é desenhada por cima dos asteroides).
3. Aperte `F5`.

> **ATENÇÃO** Os asteroides ainda atravessam a nave: o dano vem na Aula 2 com os **componentes**.



Fim da Aula 1: a nave desviando de asteroides

AULA 1

Resumo da Aula 1

- Configuramos resolução, pixel art, camadas de física e controles **pelo editor**.
- **CharacterBody2D** para a nave, **RigidBody2D** para os asteroides, **StaticBody2D** para as paredes.
- **Path2D + PathFollow2D + Timer** para gerar asteroides em volta da tela.
- **Sinais** conectados pelo editor, inclusive direto para `queue_free`.

> DICA Projeto deste ponto: pasta **projeto/checkpoint-aula1** do repositório.

AULA 2

Vida, game over, menu, dificuldade e preparação para a arte

1. GameManager
2. HUD
3. Componentes
4. Game Over
5. Menu
6. Dificuldade
7. Fundo e tema
8. Preparar a arte

AULA 2 Etapa 6.1

GameManager: o cérebro da partida (1/4)

Crie em `autoload/game_manager.gd` (Arquivos → botão direito → **Novo** → **Script**, herdando de **Node**).

autoload/game_manager.gd Copiar

```
extends Node
## GameManager (Autoload): guarda o estado da partida e o recorde.
## Qualquer script do jogo pode acessar com GameManager.alguma_coisa


## Avisa que uma partida nova começou.
signal game_started
## Avisa que o jogador morreu e a partida acabou.
signal game_over

const GAME_SCENE := "res://levels/game/game.tscn"
const MENU_SCENE := "res://ui/main_menu/main_menu.tscn"
const SAVE_PATH := "user://save.cfg"

var time_survived: float = 0.0
var best_time: float = 0.0
var is_playing: bool = false
```

AULA 2 Etapa 6.1

GameManager: o cérebro da partida (2/4)

autoload/game_manager.gd

Copiar

```
func _ready() -> void:
    load_best_time()


func _process(delta: float) -> void:
    if is_playing:
        time_survived += delta


func start_game() -> void:
    time_survived = 0.0
    is_playing = true
    game_started.emit()
```

AULA 2 Etapa 6.1

GameManager: o cérebro da partida (3/4)

autoload/game_manager.gd

Copiar

```
func end_game() -> void:
    if not is_playing:
        return
    is_playing = false
    if time_survived > best_time:
        best_time = time_survived
        save_best_time()
    game_over.emit()


func go_to_game() -> void:
    get_tree().change_scene_to_file(GAME_SCENE)


func go_to_menu() -> void:
    is_playing = false
    get_tree().change_scene_to_file(MENU_SCENE)
```

AULA 2 Etapa 6.1

GameManager: o cérebro da partida (4/4)

autoload/game_manager.gd

Copiar

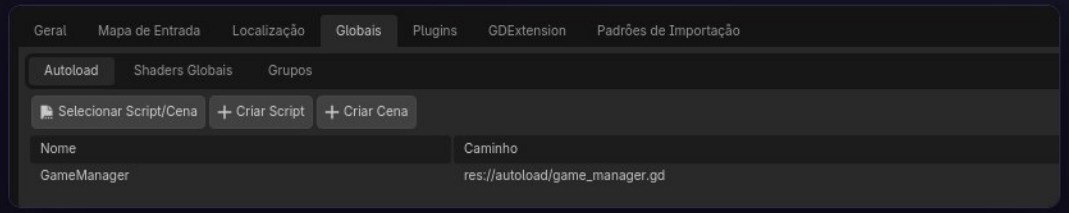
```
func save_best_time() -> void:
    var config := ConfigFile.new()
    config.set_value("score", "best_time", best_time)
    config.save(SAVE_PATH)


func load_best_time() -> void:
    var config := ConfigFile.new()
    if config.load(SAVE_PATH) == OK:
        best_time = config.get_value("score", "best_time", 0.0)
```

AULA 2 Etapa 6.2

Registrando o Autoload

1. **Projeto → Configurações do Projeto →** aba **Globais → Autoload**.
2. Clique em **Selecionar Script/Cena** e escolha `autoload/game_manager.gd`.
3. Confira o nome **GameManager** e deixe **Variável Global** habilitada.



POR QUÊ? Um **Autoload** é carregado antes de qualquer cena e nunca é apagado. Qualquer script pode escrever `GameManager.best_time`.

DICA Por isso ele guarda o que precisa sobreviver à troca de cena: recorde, estado da partida, e é ele quem troca de cena.

AULA 2 Etapa 6.3

Game: avisando que a partida começou

Anexe este script ao nó raiz **Game**. Toda vez que a cena Game carrega, o cronômetro zera e começa.

levels/game/game.gd Copiar

```
extends Node2D

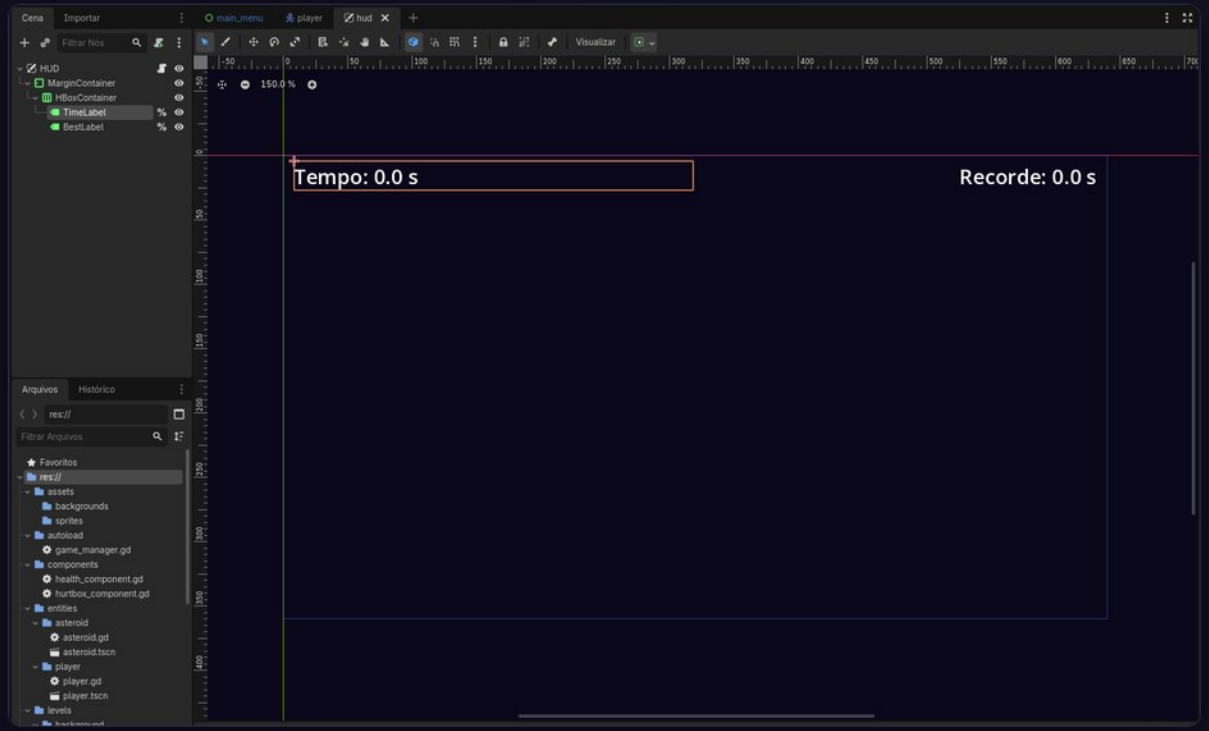

func _ready() -> void:
    GameManager.start_game()
```

AULA 2 Etapa 7.1

HUD: tempo e recorde

1. Nova cena com raiz **CanvasLayer** → **HUD**. Salve em `ui/hud/hud.tscn`
2. Filhos: **MarginContainer** (Layout **Top Wide**, margens 8/4/8) → **HBoxContainer** → 2× **Label**.
3. **TimeLabel** e **BestLabel**: Size Flags Horizontal = **Expand**. No BestLabel, Horizontal Alignment = **Right**.
4. Botão direito em cada Label → **Acessar como Nome Único** (aparece o `%`).
5. Instancie o HUD na cena Game.

> FERRAMENTA NATIVA **CanvasLayer** desenha por cima do jogo e não se mexe com a câmera. **Containers** organizam a UI sozinhos.



AULA 2 Etapa 7.2

HUD: script

Anexe ao nó raiz **HUD** (salve em `ui/hud/hud.gd`). `%TimeLabel` acha o nó pelo **nome único**: funciona mesmo se você mudar o Label de lugar na árvore.

ui/hud/hud.gd Copiar

```
extends CanvasLayer


@onready var time_label: Label = %TimeLabel
@onready var best_label: Label = %BestLabel


func _ready() -> void:
    best_label.text = "Recorde: %.1f s" % GameManager.best_time

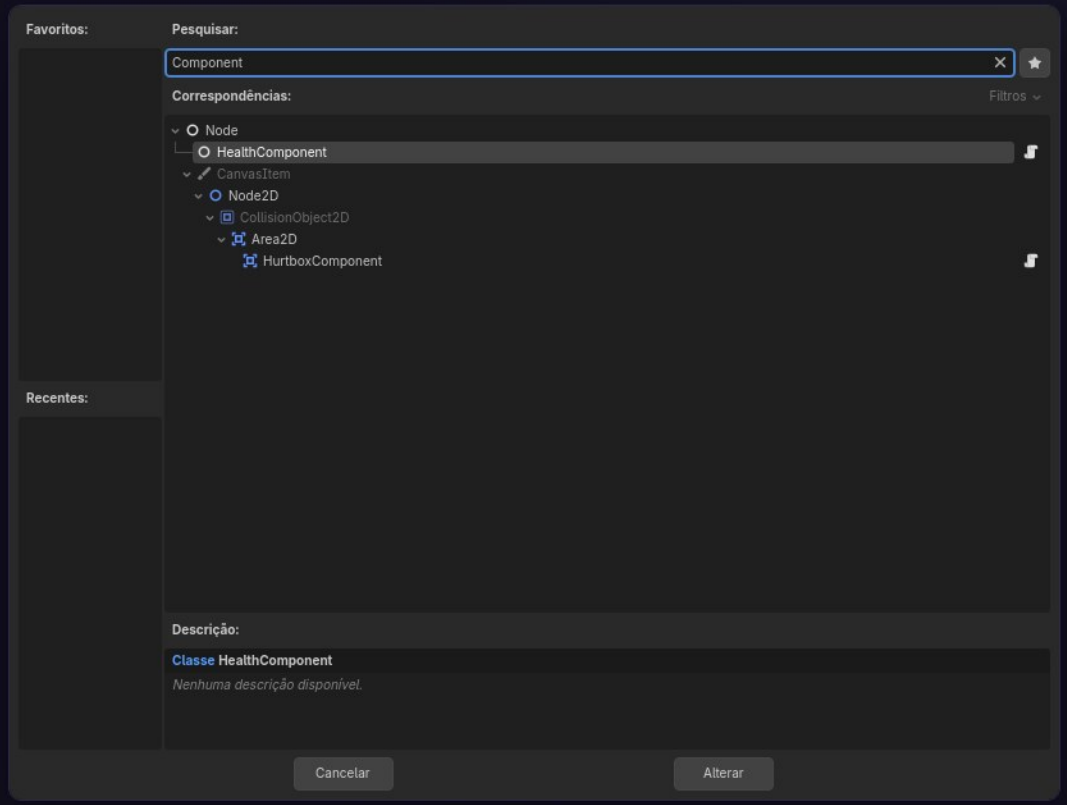

func _process(_delta: float) -> void:
    time_label.text = "Tempo: %.1f s" % GameManager.time_survived
```

AULA 2 Etapa 8.1

Componentes: por quê?

- A nave precisa de **vida** e de uma **área que recebe dano**.
- Inimigos, chefões e escudos (no futuro) também precisam disso.
- Em vez de copiar código, criamos **componentes**: nós com `class_name` que aparecem na janela **Criar Novo Nó** igual aos nós do Godot.
- **HealthComponent** (Node): guarda a vida e emite `died`.
- **HurtboxComponent** (Area2D): quando algo perigoso encosta, tira vida do HealthComponent.

> **POR QUÊ?** Componente bom não sabe quem é o dono. O HealthComponent não sabe que está numa nave: só avisa "morri".



Nossos componentes na busca de nós

AULA 2 Etapa 8.2

HealthComponent (1/2)

Crie em `components/health_component.gd`. `max_health` = 1 hoje; amanhã 3 corações é só mudar no Inspetor.

components/health_component.gd — Copiar

```gdscript
class_name HealthComponent
extends Node
## Componente de vida: guarda a vida de quem for "dono" dele e avisa quando morrer.
## Pode ser usado no player, em inimigos, em chefões...


## Emitido sempre que a vida muda (útil para barras de vida e corações).
signal health_changed(current: int, maximum: int)
## Emitido quando a vida chega a zero.
signal died

@export var max_health: int = 1

var current_health: int


func _ready() -> void:
    current_health = max_health
```

AULA 2 Etapa 8.2

HealthComponent (2/2)

components/health_component.gd

Copiar

```
func take_damage(amount: int) -> void:
    if current_health <= 0:
        return
    current_health = maxi(current_health - amount, 0)
    health_changed.emit(current_health, max_health)
    if current_health == 0:
        died.emit()


func heal(amount: int) -> void:
    current_health = mini(current_health + amount, max_health)
    health_changed.emit(current_health, max_health)
```

AULA 2 Etapa 8.3

HurtboxComponent

Aqui o sinal é conectado por código porque faz parte do próprio componente: quem usa o componente não precisa lembrar de conectar nada.

components/hurtbox_component.gd Copiar

```gdscript
class_name HurtboxComponent
extends Area2D
## Componente "área de dano": quando um corpo perigoso encosta nele,
## tira vida do HealthComponent ligado no Inspector.


@export var health_component: HealthComponent
@export var damage_per_hit: int = 1


func _ready() -> void:
    body_entered.connect(_on_body_entered)

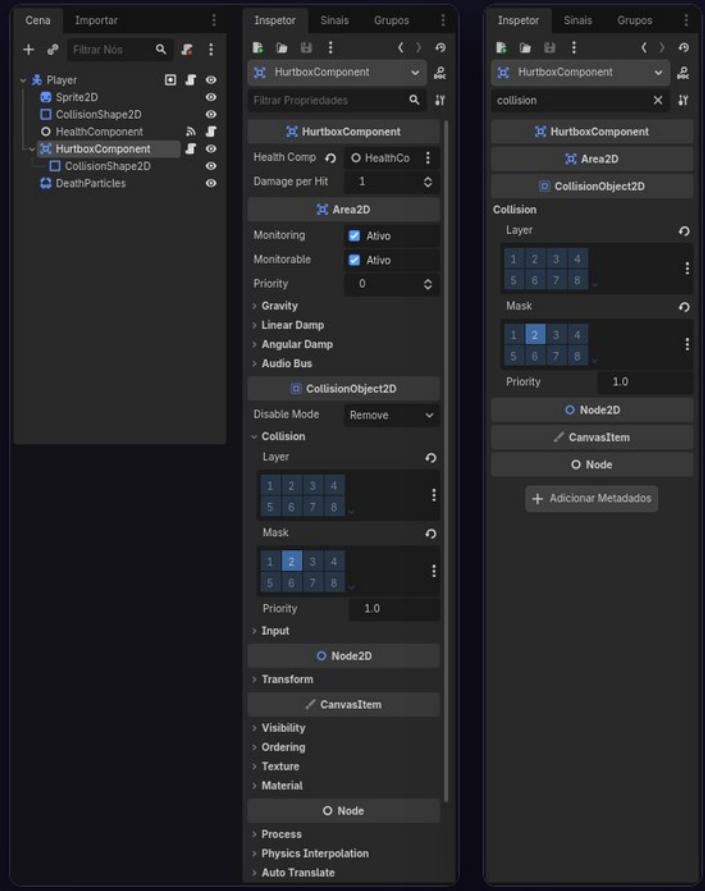

func _on_body_entered(_body: Node2D) -> void:
    health_component.take_damage(damage_per_hit)
```

AULA 2 **Etapa 8.4**

Encaixando os componentes no Player

1. Abra **player.tscn** → `Ctrl` + `A` → busque **HealthComponent** → Criar.
2. `Ctrl` + `A` → **HurtboxComponent** → dê a ele um **CollisionShape2D** (círculo raio 10).
3. No HurtboxComponent: **Health Component** → **Atribuir** → escolha o HealthComponent.
4. Collision: **Layer** nenhuma, **Mask** = 2 (asteroides).
5. No nó Player → aba **Grupos** → adicione o grupo `player` (usaremos na dificuldade).

> **FERRAMENTA NATIVA** A ligação entre componentes é feita **no Inspetor** (export), não com caminhos escritos no código.

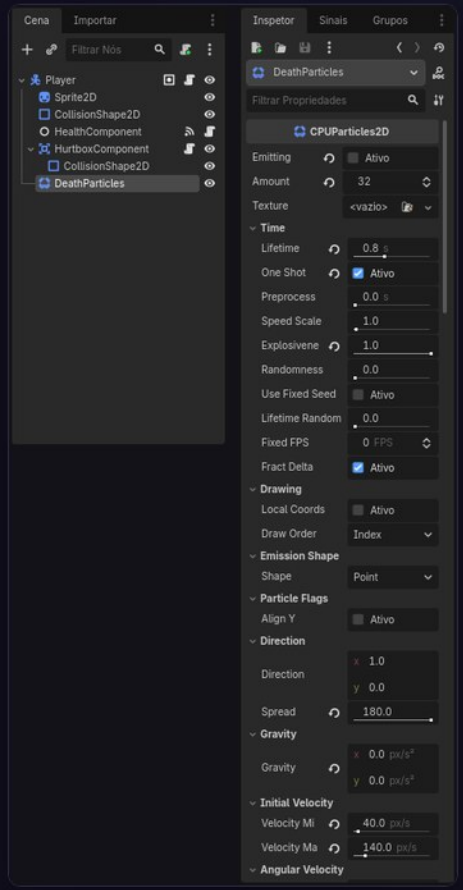


AULA 2 Etapa 8.5

Explosão com partículas

1. No Player, adicione **CPUParticles2D** → **DeathParticles**.
2. **Emitting** desligado, **One Shot** ligado, **Explosiveness** 1.
3. Amount 32, Lifetime 0.8, Spread 180°, Gravity (0, 0).
4. Initial Velocity 40–140, Damping 60–100, Scale Amount 2–4, Color laranja.

> **FERRAMENTA NATIVA** Mexa nos valores com o nó selecionado: o viewport mostra a explosão em tempo real (menu **CPUParticles2D → Restart Emission**).

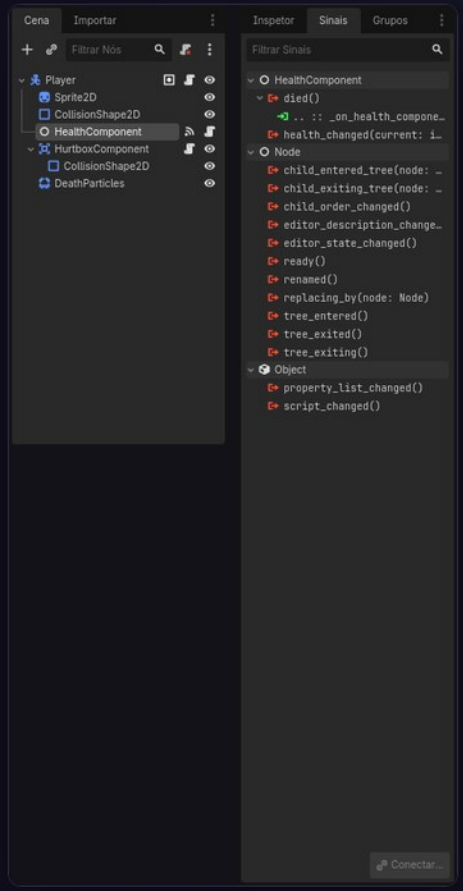


AULA 2 Etapa 8.6

Conectando a morte

1. Selecione o **HealthComponent** → aba **Sinais**.
2. Duplo clique em `died()` → escolha o nó **Player** → **Conectar**.
3. O Godot cria `_on_health_component_died` no player.gd. Complete com o código do próximo slide.

> **FERRAMENTA NATIVA** Repare: o sinal `died` que criamos no nosso script aparece na aba Sinais igualzinho aos sinais nativos.



AULA 2 Etapa 8.7

Player: script final (1/2)

Novidades: as variáveis `@onready` e a função `_on_health_component_died`. Ao morrer a nave some, explode e avisa o GameManager.

entities/player/player.gd

Copiar

```
class_name Player
extends CharacterBody2D


## Velocidade máxima da nave (pixels por segundo).
@export var speed: float = 180.0
## Quão rápido a nave acelera e freia.
@export var acceleration: float = 900.0
## Quão rápido a nave gira para a direção do movimento.
@export var rotation_speed: float = 10.0

@onready var sprite: Sprite2D = $Sprite2D
@onready var death_particles: CPUParticles2D = $DeathParticles
```

AULA 2 Etapa 8.7

Player: script final (2/2)

entities/player/player.gd

Copiar

```
func _physics_process(delta: float) -> void:
    var direction := Input.get_vector("move_left", "move_right", "move_up", "move_down")
    velocity = velocity.move_toward(direction * speed, acceleration * delta)
    move_and_slide()

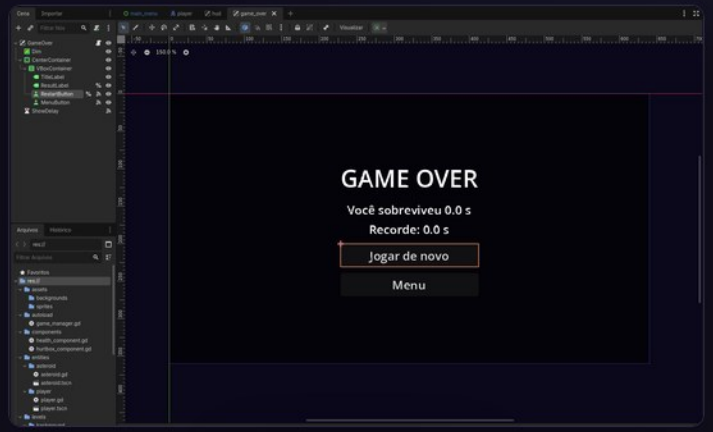
    if direction != Vector2.ZERO:
        rotation = lerp_angle(rotation, direction.angle(), rotation_speed * delta)

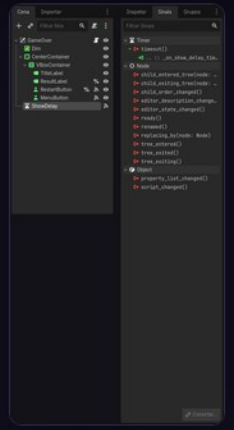

func _on_health_component_died() -> void:
    set_physics_process(false)
    sprite.hide()
    death_particles.emitting = true
    GameManager.end_game()
```

AULA 2 Etapa 9.1

Tela de Game Over

1. Nova cena **CanvasLayer** → **GameOver**, Layer = 10. Salve em `ui/game_over/game_over.tscn`
2. **ColorRect** (Dim): Layout **Full Rect**, cor preta com alpha 0.6.
3. **CenterContainer** (Full Rect) → **VBoxContainer** (Separation 8) → Labels **TitleLabel** ("GAME OVER", tamanho 32) e **ResultLabel** (%) e Buttons **RestartButton** (%) e **MenuButton**.
4. Um **Timer ShowDelay** (Wait Time 1, One Shot) para dar tempo de ver a explosão.
5. Anexe um script ao **GameOver** e conecte pela aba Sinais: `pressed` dos 2 botões e `timeout` do ShowDelay.
6. Instancie o GameOver na cena Game e clique no **olho** dele para escondê-lo no editor (o script mostra a tela na hora certa).





AULA 2 Etapa 9.2

Game Over: script (1/2)

Anexe ao nó raiz **GameOver** (`ui/game_over/game_over.gd`) **antes** de conectar os sinais. A tela começa escondida e só aparece quando o GameManager avisa `game_over` . `grab_focus()` deixa jogar de novo apertando Enter ou no controle.

ui/game_over/game_over.gd

Copiar

```
extends CanvasLayer


@onready var result_label: Label = %ResultLabel
@onready var restart_button: Button = %RestartButton
@onready var show_delay: Timer = $ShowDelay


func _ready() -> void:
    hide()
    GameManager.game_over.connect(_on_game_over)


func _on_game_over() -> void:
    # Espera um pouquinho para dar tempo de ver a explosão.
    show_delay.start()
```

AULA 2 Etapa 9.2

Game Over: script (2/2)

ui/game_over/game_over.gd

Copiar

```
func _on_show_delay_timeout() -> void:
    var text := "Você sobreviveu %.1f s\nRecorde: %.1f s"
    result_label.text = text % [GameManager.time_survived, GameManager.best_time]
    show()
    restart_button.grab_focus()


func _on_restart_button_pressed() -> void:
    GameManager.go_to_game()

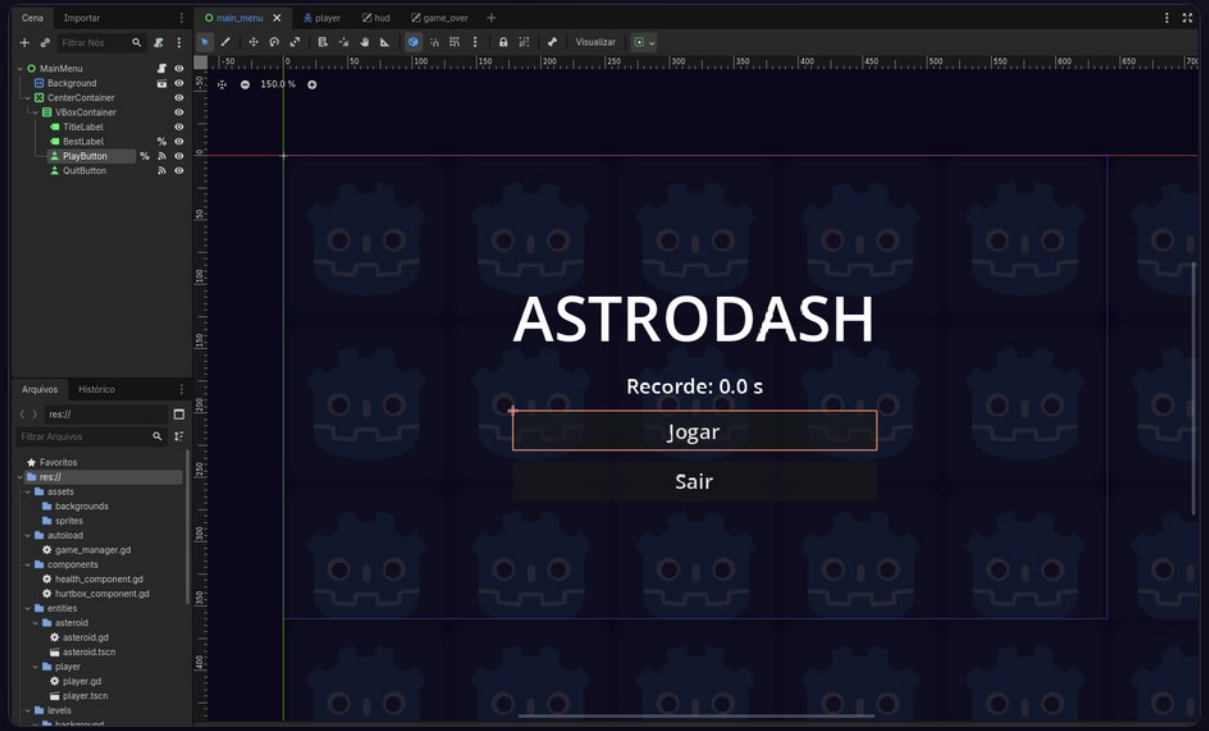

func _on_menu_button_pressed() -> void:
    GameManager.go_to_menu()
```

AULA 2 Etapa 10.1

Menu inicial

1. Nova cena **Interface de Usuário** (Control) → **MainMenu**. Salve em `ui/main_menu/main_menu.tscn`
2. **CenterContainer** (Full Rect) → **VBoxContainer** → **TitleLabel** ("ASTRODASH", tamanho 48), **BestLabel** (%), **PlayButton** (%) e **QuitButton**.
3. Anexe um script ao **MainMenu** e conecte o `pressed` dos dois botões a ele.
4. Botão direito em `main_menu.tscn` → **Definir como Cena Principal**.

> FERRAMENTA NATIVA Use **Theme Overrides** → **Font Sizes** no Inspetor para aumentar só o título.



AULA 2 Etapa 10.2

Menu: script

Anexe ao nó raiz **MainMenu** (`ui/main_menu/main_menu.gd`) antes de conectar os botões.

ui/main_menu/main_menu.gd Copiar

```
extends Control


@onready var best_label: Label = %BestLabel
@onready var play_button: Button = %PlayButton


func _ready() -> void:
    best_label.text = "Recorde: %.1f s" % GameManager.best_time
    play_button.grab_focus()


func _on_play_button_pressed() -> void:
    GameManager.go_to_game()

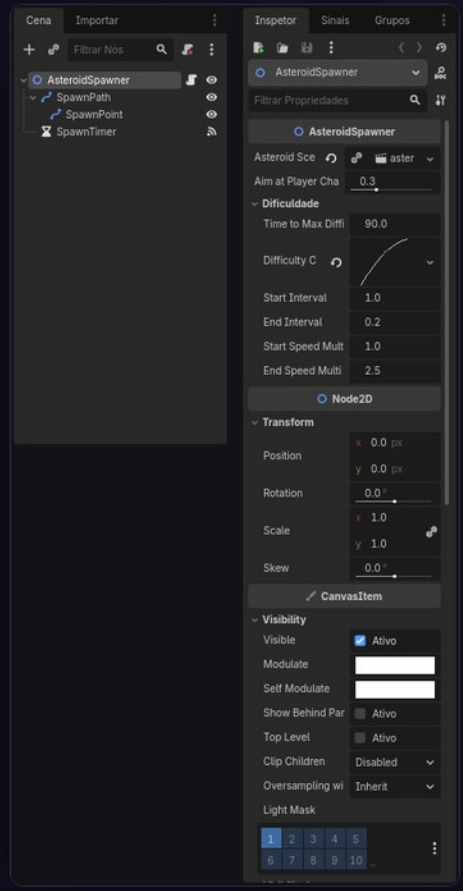

func _on_quit_button_pressed() -> void:
    get_tree().quit()
```

AULA 2 Etapa 11.1

Dificuldade com uma Curve

- Com o tempo, os asteroides nascem **mais rápido** e voam **mais rápido**.
- Em vez de fórmulas, usamos um recurso **Curve** exportado: você **desenha** a curva de dificuldade no Inspetor.
- Eixo X = progresso do tempo (0 → 1 em **Time To Max Difficulty** segundos). Eixo Y = dificuldade (0 → 1).
- Parte dos asteroides (**Aim At Player Chance**) mira na nave, usando o grupo `player`: ficar parado num canto não salva ninguém.
- O grupo **Dificuldade** no Inspetor vem do `@export_group`.

> **FERRAMENTA NATIVA** Clique na curva no Inspetor para editar: arraste os pontos, clique com o direito para adicionar. Teste curvas diferentes!



AULA 2 Etapa 11.2

Spawner: script final (1/3)

Selecione o AsteroidSpawner → **Difficulty Curve** → **Novo Curve** → desenhe uma curva que sobe rápido no começo e suaviza no fim.

systems/asteroid_spawner/asteroid_spawner.gd

Copiar

```
class_name AsteroidSpawner
extends Node2D
## Cria asteroides em volta da tela, cruzando a tela em direções aleatórias.
## A dificuldade sobe com o tempo seguindo a curva do Inspector.


@export var asteroid_scene: PackedScene
## Chance (0 a 1) de um asteroide vir mirando direto no player.
@export_range(0.0, 1.0) var aim_at_player_chance: float = 0.3

@export_group("Dificuldade")
## Tempo (em segundos) até chegar na dificuldade máxima.
@export var time_to_max_difficulty: float = 90.0
## X = progresso do tempo (0 a 1), Y = dificuldade (0 a 1).
@export var difficulty_curve: Curve
@export var start_interval: float = 1.0
@export var end_interval: float = 0.2
@export var start_speed_multiplier: float = 1.0
@export var end_speed_multiplier: float = 2.5

var speed_multiplier: float = 1.0
```

AULA 2 Etapa 11.2

Spawner: script final (2/3)

systems/asteroid_spawner/asteroid_spawner.gd Copiar

```gdscript
@onready var spawn_point: PathFollow2D = $SpawnPath/SpawnPoint
@onready var spawn_timer: Timer = $SpawnTimer


func _ready() -> void:
    GameManager.game_over.connect(spawn_timer.stop)


func _on_spawn_timer_timeout() -> void:
    _update_difficulty()

    spawn_point.progress_ratio = randf()
    var asteroid: Asteroid = asteroid_scene.instantiate()
    add_child(asteroid)
    asteroid.global_position = spawn_point.global_position

    var target := _pick_target()
    asteroid.launch(asteroid.global_position.direction_to(target), speed_multiplier)
```

AULA 2 Etapa 11.2

Spawner: script final (3/3)

systems/asteroid_spawner/asteroid_spawner.gd Copiar

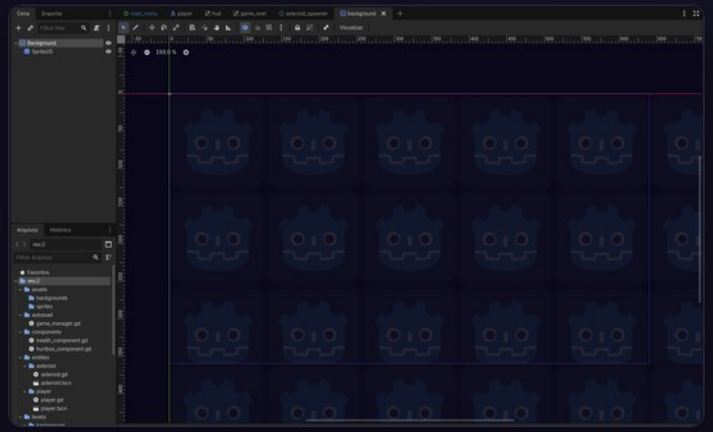
```
func _update_difficulty() → void:
    var progress := clampf(GameManager.time_survived / time_to_max_difficulty, 0.0, 1.0)
    var difficulty := difficulty_curve.sample(progress) if difficulty_curve else progress
    spawn_timer.wait_time = lerpf(start_interval, end_interval, difficulty)
    speed_multiplier = lerpf(start_speed_multiplier, end_speed_multiplier, difficulty)


## Escolhe para onde o asteroide vai: às vezes no player, senão um ponto qualquer da tela.
func _pick_target() → Vector2:
    var player := get_tree().get_first_node_in_group("player") as Node2D
    if player and randf() < aim_at_player_chance:
        return player.global_position
    var screen := get_viewport_rect().size
    return Vector2(randf_range(0, screen.x), randf_range(0, screen.y))
```

AULA 2 Etapa 12.1

Fundo animado com Parallax2D

1. Nova cena com raiz **Parallax2D** → **Background**. Salve em `levels/background/background.tscn`
2. Parallax2D: **Repeat Size** = (640, 360), **Autoscroll** = (-20, 0).
3. Filho **Sprite2D**: Texture = icon.svg, **Centered** desligado, **Region** → **Enabled** e Rect = (0, 0, 640, 360).
4. **CanvasItem** → **Texture** → **Repeat** = `Enabled` (o ícone se repete). **Modulate** alpha ≈ 30.
5. Instancie o Background como **primeiro** filho do Game e do MainMenu.



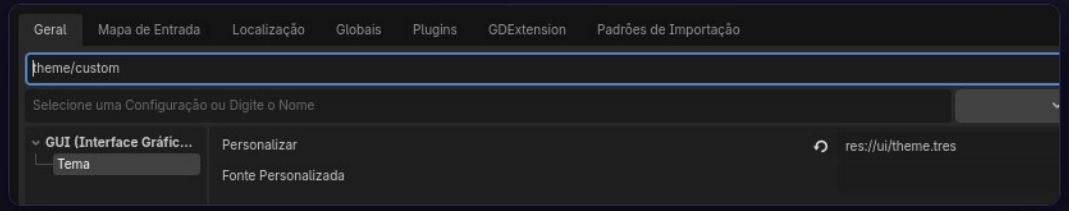


FERRAMENTA NATIVA O **Parallax2D** faz o fundo andar e se repetir infinitamente, sem uma linha de código. Na 2ª parte é só trocar a textura por um fundo de estrelas 640×360.

AULA 2 Etapa 12.2

Tema global da interface

1. Arquivos → botão direito em `ui/` → **Novo** → **Recurso** → **Theme** → salve `ui/theme.tres`
2. No Inspetor do tema: **Default Font Size** = 16.
3. Configurações do Projeto → busque `theme/custom` → **GUI** → **Tema** → **Personalizado** = `res://ui/theme.tres`.



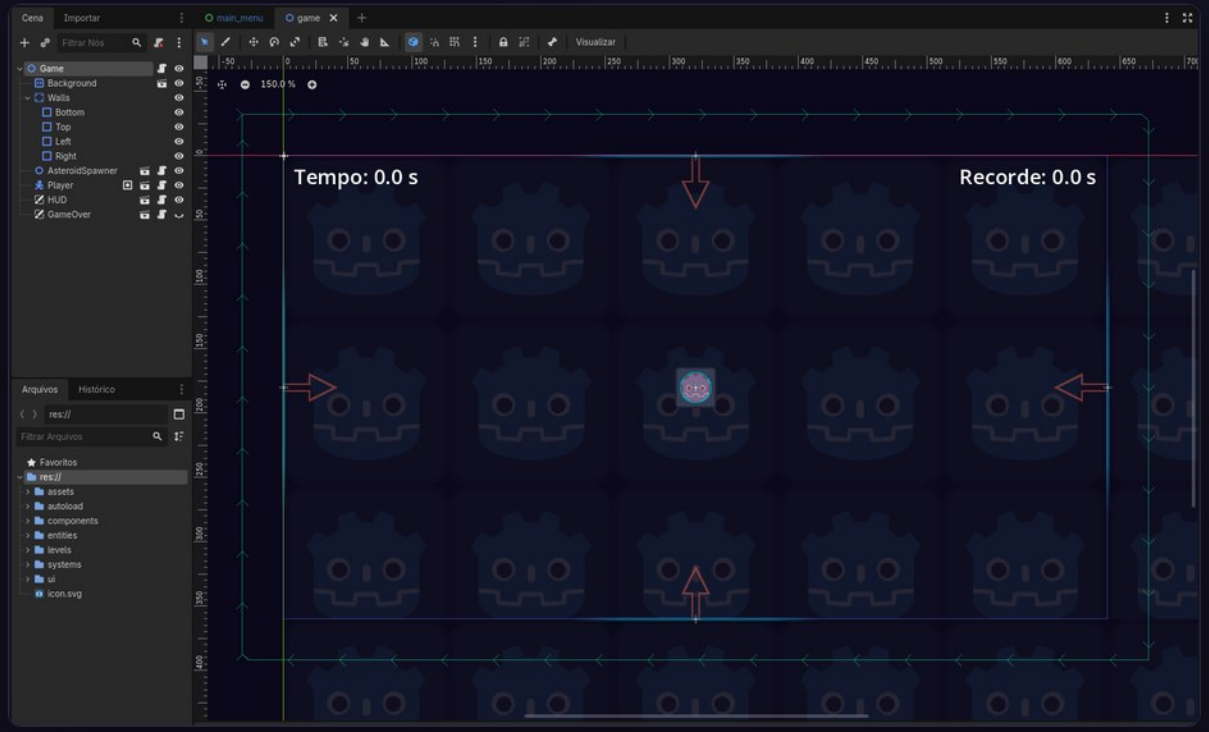
> **POR QUÊ?** Na 2ª parte, colocar uma fonte pixel art no tema muda **todos** os textos do jogo de uma vez.

AULA 2 Etapa 12.3

A cena Game completa

NÓ	O QUE FAZ
Background	Fundo animado (Parallax2D)
Walls	Prende a nave dentro da tela
AsteroidSpawner	Cria asteroides e controla a dificuldade
Player	A nave, com seus componentes
HUD	Tempo e recorde
GameOver	Aparece quando o player morre

> **DICA** A ordem na árvore é a ordem de desenho: de cima (atrás) para baixo (na frente).



AULA 2 Etapa 12.4

Jogo pronto!

- Menu → Jogar → sobreviver → explosão → Game Over → jogar de novo.
- O recorde fica salvo em `user://save.cfg` pelo **ConfigFile**, então continua lá ao fechar o jogo.



Menu



Game Over

PREPARANDO PARA A ARTE

2ª parte do curso: LibreSprite

1. Onde cada arte entra
2. Tamanhos
3. Como trocar

ARTE Etapa 13.1

Onde cada arte entra

ARTE	TAMANHO SUGERIDO	ONDE TROCAR
Nave	32×32 (apontando para a **direita**)	`entities/player/player.tscn` → Sprite2D → Texture
Asteroide	32×32 (pode fazer 2-3 variações)	`entities/asteroid/asteroid.tscn` → Sprite2D → Texture
Fundo	640×360 (bordas que se encaixam)	`levels/background/background.tscn` → Sprite2D → Texture
Partícula	4×4 ou 8×8	Player → DeathParticles → Texture
Fonte	Fonte pixel (.ttf)	`ui/theme.tres` → Default Font

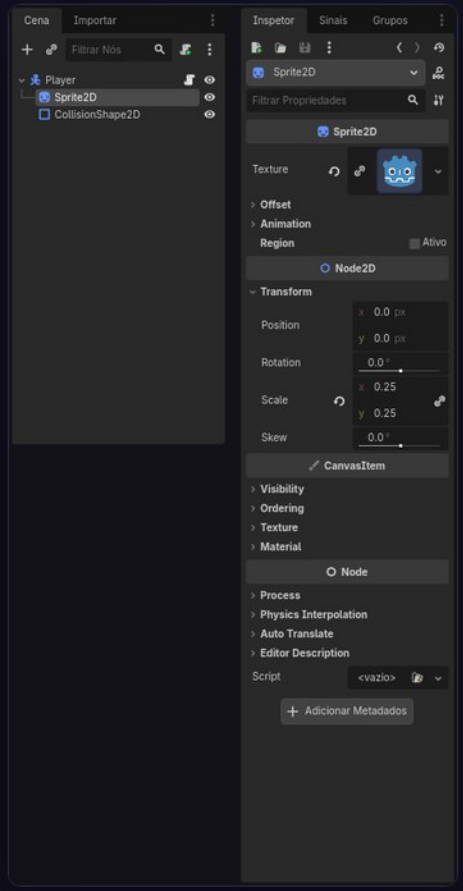
DICA Salve os PNGs exportados do LibreSprite em `assets/sprites/` e `assets/backgrounds/`. O filtro **Nearest** já está ligado no projeto todo.

ARTE Etapa 13.2

Como trocar uma arte

1. No LibreSprite: **Arquivo → Exportar** → PNG na pasta `assets/sprites/` do projeto.
2. Volte ao Godot: o arquivo aparece sozinho nos Arquivos.
3. Abra a cena (ex.: player.tscn) → selecione o **Sprite2D**.
4. Arraste o PNG para **Texture**.
5. Volte o **Scale** para `(1, 1)` (o 0.25 era só para o ícone de 128px).
6. Se a arte tiver outro tamanho, ajuste o raio do **CollisionShape2D**.

> POR QUÊ? Como o asteroide é uma cena só, trocar a arte dele muda **todos** os asteroides do jogo. Essa é a vantagem de separar em cenas.



ARTE

Desafios para quem terminar antes

1. **3 vidas**: `max_health = 3` + mostrar corações no HUD usando o sinal `health_changed`.
2. **Invencibilidade** por 1 segundo depois de levar dano (Timer + piscar com **AnimationPlayer**).
3. **Asteroides de tamanhos diferentes**: crie **Cenas Herdadas** de asteroid.tscn com outro sprite e colisão.
4. **Power-up de escudo**: um Area2D que chama `heal()` no HealthComponent.
5. **Som**: **AudioStreamPlayer** para a explosão e música no menu.
6. **Tremer a tela**: uma **Camera2D** com offset aleatório quando o player morre.



Bom jogo!

Próximo passo: abrir o LibreSprite e dar cara própria ao AstroDash.

Todos os códigos estão nos slides seguintes (Apêndice) e no repositório.

APÊNDICE · AULA 1

entities/player/player.gd

entities/player/player.gd

Copiar

```
class_name Player
extends CharacterBody2D


## Velocidade máxima da nave (pixels por segundo).
@export var speed: float = 180.0
## Quão rápido a nave acelera e freia.
@export var acceleration: float = 900.0
## Quão rápido a nave gira para a direção do movimento.
@export var rotation_speed: float = 10.0


func _physics_process(delta: float) -> void:
    var direction := Input.get_vector("move_left", "move_right", "move_up", "move_down")
    velocity = velocity.move_toward(direction * speed, acceleration * delta)
    move_and_slide()

    if direction != Vector2.ZERO:
        rotation = lerp_angle(rotation, direction.angle(), rotation_speed * delta)
```

APÊNDICE · AULA 1

entities/asteroid/asteroid.gd

entities/asteroid/asteroid.gd

Copiar

```
class_name Asteroid
extends RigidBody2D


@export var min_speed: float = 60.0
@export var max_speed: float = 120.0
## Velocidade máxima de giro (radianos por segundo).
@export var max_spin: float = 3.0


## Lança o asteroide em uma direção. Quem chama é o AsteroidSpawner.
func launch(direction: Vector2, speed_multiplier: float = 1.0) -> void:
    linear_velocity = direction * randf_range(min_speed, max_speed) * speed_multiplier
    angular_velocity = randf_range(-max_spin, max_spin)
```

APÊNDICE · AULA 1

systems/asteroid_spawner/asteroid_spawner.gd

systems/asteroid_spawner/asteroid_spawner.gd

Copiar

```gdscript
class_name AsteroidSpawner
extends Node2D
## Cria asteroides em volta da tela, cruzando a tela em direções aleatórias.


@export var asteroid_scene: PackedScene

@onready var spawn_point: PathFollow2D = $SpawnPath/SpawnPoint
```

```gdscript
func _on_spawn_timer_timeout() -> void:
    spawn_point.progress_ratio = randf()
    var asteroid: Asteroid = asteroid_scene.instantiate()
    add_child(asteroid)
    asteroid.global_position = spawn_point.global_position

    var target := _pick_target()
    asteroid.launch(asteroid.global_position.direction_to(target))


## Escolhe um ponto qualquer da tela para o asteroide atravessar.
func _pick_target() -> Vector2:
    var screen := get_viewport_rect().size
    return Vector2(randf_range(0, screen.x), randf_range(0, screen.y))
```

APÊNDICE · FINAL

autoload/game_manager.gd

autoload/game_manager.gd

Copiar

```gdscript
extends Node
## GameManager (Autoload): guarda o estado da partida e o recorde.
## Qualquer script do jogo pode acessar com GameManager.alguma_coisa


## Avisa que uma partida nova começou.
signal game_started
## Avisa que o jogador morreu e a partida acabou.
signal game_over

const GAME_SCENE := "res://levels/game/game.tscn"
const MENU_SCENE := "res://ui/main_menu/main_menu.tscn"
const SAVE_PATH := "user://save.cfg"

var time_survived: float = 0.0
var best_time: float = 0.0
var is_playing: bool = false


func _ready() -> void:
    load_best_time()


func _process(delta: float) -> void:
    if is_playing:
        time_survived += delta


func start_game() -> void:
    time_survived = 0.0
    is_playing = true
    game_started.emit()


func end_game() -> void:
    if not is_playing:
        return
    is_playing = false
    if time_survived > best_time:
        best_time = time_survived
        save_best_time()
    game_over.emit()


func go_to_game() -> void:
    get_tree().change_scene_to_file(GAME_SCENE)


func go_to_menu() -> void:
    is_playing = false
    get_tree().change_scene_to_file(MENU_SCENE)


func save_best_time() -> void:
    var config := ConfigFile.new()
    config.set_value("score", "best_time", best_time)
    config.save(SAVE_PATH)


func load_best_time() -> void:
    var config := ConfigFile.new()
    if config.load(SAVE_PATH) == OK:
        best_time = config.get_value("score", "best_time", 0.0)
```

APÊNDICE · FINAL

components/health_component.gd

components/health_component.gd

Copiar

```gdscript
class_name HealthComponent
extends Node
## Componente de vida: guarda a vida de quem for "dono" dele e avisa quando morrer.
## Pode ser usado no player, em inimigos, em chefões...


## Emitido sempre que a vida muda (útil para barras de vida e corações).
signal health_changed(current: int, maximum: int)
## Emitido quando a vida chega a zero.
signal died

@export var max_health: int = 1

var current_health: int

func _ready() -> void:
    current_health = max_health


func take_damage(amount: int) -> void:
    if current_health <= 0:
        return
    current_health = maxi(current_health - amount, 0)
    health_changed.emit(current_health, max_health)
    if current_health == 0:
        died.emit()


func heal(amount: int) -> void:
    current_health = mini(current_health + amount, max_health)
    health_changed.emit(current_health, max_health)
```

APÊNDICE · FINAL

components/hurtbox_component.gd

components/hurtbox_component.gd Copiar

```
class_name HurtboxComponent
extends Area2D
## Componente "área de dano": quando um corpo perigoso encosta nele,
## tira vida do HealthComponent ligado no Inspector.


@export var health_component: HealthComponent
@export var damage_per_hit: int = 1


func _ready() -> void:
    body_entered.connect(_on_body_entered)


func _on_body_entered(_body: Node2D) -> void:
    health_component.take_damage(damage_per_hit)
```

APÊNDICE · FINAL

entities/player/player.gd

entities/player/player.gd

Copiar

```gdscript
class_name Player
extends CharacterBody2D


## Velocidade máxima da nave (pixels por segundo).
@export var speed: float = 180.0
## Quão rápido a nave acelera e freia.
@export var acceleration: float = 900.0
## Quão rápido a nave gira para a direção do movimento.
@export var rotation_speed: float = 10.0

@onready var sprite: Sprite2D = $Sprite2D
@onready var death_particles: CPUParticles2D = $DeathParticles
```

```gdscript
func _physics_process(delta: float) -> void:
    var direction := Input.get_vector("move_left", "move_right", "move_up", "move_down")
    velocity = velocity.move_toward(direction * speed, acceleration * delta)
    move_and_slide()

    if direction != Vector2.ZERO:
        rotation = lerp_angle(rotation, direction.angle(), rotation_speed * delta)


func _on_health_component_died() -> void:
    set_physics_process(false)
    sprite.hide()
    death_particles.emitting = true
    GameManager.end_game()
```

APÊNDICE · FINAL

entities/asteroid/asteroid.gd

entities/asteroid/asteroid.gd

Copiar

```
class_name Asteroid
extends RigidBody2D


@export var min_speed: float = 60.0
@export var max_speed: float = 120.0
## Velocidade máxima de giro (radianos por segundo).
@export var max_spin: float = 3.0


## Lança o asteroide em uma direção. Quem chama é o AsteroidSpawner.
func launch(direction: Vector2, speed_multiplier: float = 1.0) -> void:
    linear_velocity = direction * randf_range(min_speed, max_speed) * speed_multiplier
    angular_velocity = randf_range(-max_spin, max_spin)
```

APÊNDICE · FINAL

systems/asteroid_spawner/asteroid_spawner.gd

systems/asteroid_spawner/asteroid_spawner.gd

Copiar

```gdscript
class_name AsteroidSpawner
extends Node2D
## Cria asteroides em volta da tela, cruzando a tela em direções aleatórias.
## A dificuldade sobe com o tempo seguindo a curva do Inspector.


@export var asteroid_scene: PackedScene
## Chance (0 a 1) de um asteroide vir mirando direto no player.
@export_range(0.0, 1.0) var aim_at_player_chance: float = 0.3

@export_group("Dificuldade")
## Tempo (em segundos) até chegar na dificuldade máxima.
@export var time_to_max_difficulty: float = 90.0
## X = progresso do tempo (0 a 1), Y = dificuldade (0 a 1).
@export var difficulty_curve: Curve
@export var start_interval: float = 1.0
@export var end_interval: float = 0.2
@export var start_speed_multiplier: float = 1.0
@export var end_speed_multiplier: float = 2.5

var speed_multiplier: float = 1.0

@onready var spawn_point: PathFollow2D = $SpawnPath/SpawnPoint
@onready var spawn_timer: Timer = $SpawnTimer


func _ready() -> void:
    GameManager.game_over.connect(spawn_timer.stop)


func _on_spawn_timer_timeout() -> void:
    _update_difficulty()

    spawn_point.progress_ratio = randf()
    var asteroid: Asteroid = asteroid_scene.instantiate()
    add_child(asteroid)
    asteroid.global_position = spawn_point.global_position

    var target := _pick_target()
    asteroid.launch(asteroid.global_position.direction_to(target), speed_multiplier)


func _update_difficulty() -> void:
    var progress := clampf(GameManager.time_survived / time_to_max_difficulty, 0.0, 1.0)
    var difficulty := difficulty_curve.sample(progress) if difficulty_curve else progress
    spawn_timer.wait_time = lerpf(start_interval, end_interval, difficulty)
    speed_multiplier = lerpf(start_speed_multiplier, end_speed_multiplier, difficulty)


## Escolhe para onde o asteroide vai: às vezes no player, senão um ponto qualquer da tela.
func _pick_target() -> Vector2:
    var player := get_tree().get_first_node_in_group("player") as Node2D
    if player and randf() < aim_at_player_chance:
        return player.global_position
    var screen := get_viewport_rect().size
    return Vector2(randf_range(0, screen.x), randf_range(0, screen.y))
```

APÊNDICE · FINAL

levels/game/game.gd

levels/game/game.gd

Copiar

```
extends Node2D


func _ready() -> void:
    GameManager.start_game()
```

APÊNDICE · FINAL

ui/hud/hud.gd

ui/hud/hud.gd Copiar

```
extends CanvasLayer


@onready var time_label: Label = %TimeLabel
@onready var best_label: Label = %BestLabel


func _ready() -> void:
    best_label.text = "Recorde: %.1f s" % GameManager.best_time


func _process(_delta: float) -> void:
    time_label.text = "Tempo: %.1f s" % GameManager.time_survived
```

APÊNDICE · FINAL

ui/game_over/game_over.gd

ui/game_over/game_over.gd

Copiar

```gdscript
extends CanvasLayer


@onready var result_label: Label = %ResultLabel
@onready var restart_button: Button = %RestartButton
@onready var show_delay: Timer = $ShowDelay


func _ready() -> void:
    hide()
    GameManager.game_over.connect(_on_game_over)


func _on_game_over() -> void:
    # Espera um pouquinho para dar tempo de ver a explosão.
    show_delay.start()


func _on_show_delay_timeout() -> void:
    var text := "Você sobreviveu %.1f s\nRecorde: %.1f s"
    result_label.text = text % [GameManager.time_survived, GameManager.best_time]
    show()
    restart_button.grab_focus()


func _on_restart_button_pressed() -> void:
    GameManager.go_to_game()


func _on_menu_button_pressed() -> void:
    GameManager.go_to_menu()
```

APÊNDICE · FINAL

ui/main_menu/main_menu.gd

ui/main_menu/main_menu.gd

Copiar

```
extends Control


@onready var best_label: Label = %BestLabel
@onready var play_button: Button = %PlayButton


func _ready() -> void:
    best_label.text = "Recorde: %.1f s" % GameManager.best_time
    play_button.grab_focus()


func _on_play_button_pressed() -> void:
    GameManager.go_to_game()


func _on_quit_button_pressed() -> void:
    get_tree().quit()
```